

Database Research needs an Abstract Relational Query Language

Wolfgang Gatterbauer

Thanks among others to: Diandre Sabale, Cody Dunne, Jonathan Danaparamita, Mirek Riedewald, Jiahui Zhang

NorthEast Database Day 2026 (updated March 2026)

<https://RelationalDiagrams.com>



A discussion of the evolution of the database industry over the past half century, and why the relational database concepts introduced by E.F. Codd have proven to be so resilient over several decades.

BY DONALD CHAMBERLIN

50 Years of Queries

CACM 2024. <https://doi.org/10.1145/3649887>

A Formal Semantics of SQL Queries, Its Validation, and Applications

Paolo Guagliardo
School of Informatics
University of Edinburgh
pguaglia@inf.ed.ac.uk

Leonid Libkin
School of Informatics
University of Edinburgh
libkin@inf.ed.ac.uk

PVLDB 2017

"the (SQL) Standard... cannot serve as a formal semantics... because it is ... inherently ambiguous."

1 Can SQL really serve as reference language?

A discussion of the evolution of the database industry over the past half century, and why the relational database concepts introduced by E.F. Codd have proven to be so resilient over several decades.

BY DONALD CHAMBERLIN

50 Years of Queries

CACM 2024. <https://doi.org/10.1145/3649887>

A Formal Semantics of SQL Queries, Its Validation, and Applications

Paolo Guagliardo
School of Informatics
University of Edinburgh
pguaglia@inf.ed.ac.uk

Leonid Libkin
School of Informatics
University of Edinburgh
libkin@inf.ed.ac.uk

PVLDB 2017

"if we want to capture the meaning of SQL queries in the real world we cannot just pretend that nulls do not exist."

Rel: A Programming Language for Relational Data

Molham Aref
RelationalAI
molham.aref@relational.ai

Paolo Guagliardo
University of Edinburgh
paolo.guagliardo@ed.ac.uk

George Kastrinis
RelationalAI
george.kastrinis@relational.ai

Leonid Libkin
RelationalAI & Univ of Edinburgh
leonid.libkin@relational.ai

Victor Marsault
LIGM, Univ. Gustave Eiffel, CNRS
victor.marsault@univ-eiffel.fr

Wim Martens
RelationalAI & University of Bayreuth
wim.martens@relational.ai

Mary McGrath
RelationalAI
mary.mcgrath@skylight.digital

Filip Murlak
University of Warsaw
f.murlak@uw.edu.pl

Nathaniel Nystrom
RelationalAI
nate.nystrom@relational.ai

Liat Peterfreund
Hebrew University
liat.peterfreund@mail.huji.ac.il

Allison Rogers
RelationalAI
allison.rogers@relational.ai

Cristina Sirangelo
Université Paris Cité, CNRS, IRIF
cristina@irif.fr

Domagoj Vrgoč
PUC Chile
vrdomagoj@uc.cl

David Zhao
RelationalAI
david.zhao@relational.ai

Abdul Zreika
RelationalAI
abdul.zreika@relational.ai

SIGMOD 2025

".. the pure relational model ... has neither multiplicities (bags) nor nulls..."

2 Set or bag semantics as reference? Null values or not?

"algebra with its explicit operator ordering is a good foundation for a modern query language." "nicer ... way to express queries"

A Critique of Modern SQL And A Proposal Towards A Simple and Expressive Query Language

Thomas Neumann
Technische Universität München
neumann@in.tum.de

Viktor Leis
Technische Universität München
leis@in.tum.de

CIDR 2024

"(SQL's) 'inside-out' structure makes ... SQL logic difficult." "pipe syntax ... can make the query ... easier to understand."

SQL Has Problems. We Can Fix Them: Pipe Syntax In SQL

Jeff Shute Google, Inc.	Shannon Bales Google, Inc.	Matthew Brown Google, Inc.	Jean-Daniel Browne Google, Inc.	Brandon Dolphin Google, Inc.
Romit Kudtarkar Google, Inc.	Andrey Litvinov Google, Inc.	Jingchi Ma Google, Inc.	John Morcos Google, Inc.	Michael Shen Google, Inc.
	David Wilhite Google, Inc.	Xi Wu Google, Inc.	Lulan Yu Google, Inc.	

PVLDB 2024

3 Linear dataflow is easier to understand than inside-out.

A Critique of Modern SQL And A Proposal Towards A Simple and Expressive Query Language

Thomas Neumann
Technische Universität München
neumann@in.tum.de

Viktor Leis
Technische Universität München
leis@in.tum.de

CIDR 2024

"modularity ... SaneQL allows reusing logic across queries, which is not traditionally done in SQL"

"SQL's syntactic structure for composing queries using those operations is terrible."

SQL Has Problems. We Can Fix Them: Pipe Syntax In SQL

Jeff Shute Google, Inc.	Shannon Bales Google, Inc.	Matthew Brown Google, Inc.	Jean-Daniel Browne Google, Inc.	Brandon Dolphin Google, Inc.
Romit Kudtarkar Google, Inc.	Andrey Litvinov Google, Inc.	Jingchi Ma Google, Inc.	John Morcos Google, Inc.	Michael Shen Google, Inc.
	David Wilhite Google, Inc.	Xi Wu Google, Inc.	Lulan Yu Google, Inc.	

PVLDB 2024

4 Linear dataflow allows modularity, i.e. composing queries

Pig Latin: A Not-So-Foreign Language for Data Processing

Christopher Olston^{*}
Yahoo! Research

Benjamin Reed[†]
Yahoo! Research

Utkarsh Srivastava[‡]
Yahoo! Research

Ravi Kumar[§]
Yahoo! Research

Andrew Tomkins[¶]
Yahoo! Research

SIGMOD 2008

"Dataflow Language ... making it easier for programmers to understand ... their data processing task"

DFQL: Dataflow query language for relational databases

Gard J. Clark, C. Thomas Wu^{*}

Naval Postgraduate School, Department of Computer Science, Code CS, Monterey, CA 93943, USA 1994

"humans think of complex problems in a sequential fashion"

"subjects using the more procedural language wrote difficult queries better than subjects using the less procedural language."

Human Factors Comparison of a Procedural and a Nonprocedural Query Language

CHARLES WELTY
University of Southern Maine
and

DAVID W. STEMPLER
University of Massachusetts

1981

5 How can we shed new light into the decades-old pipe/dataflow debate?

Pig Latin: A Not-So-Foreign Language for Data Processing

Christopher Olston^{*}
Yahoo! Research

Benjamin Reed[†]
Yahoo! Research

Utkarsh Srivastava[‡]
Yahoo! Research

Ravi Kumar[§]
Yahoo! Research

Andrew Tomkins[¶]
Yahoo! Research

SIGMOD 2008

On The Reasonable Effectiveness of Relational Diagrams^{*}

Explaining Relational Query Patterns and the Pattern Expressiveness of Relational Languages

WOLFGANG GATTERBAUER,  Northeastern University, USA

CODY DUNNE,  Northeastern University, USA

SIGMOD 2024, SIGMOD record 2025

"relational calculus can express a larger class of patterns than the basic operators of relational algebra."

6 Can we have both: easy-to-understand and full pattern expressiveness?

The Cambridge Report on Database Research

Anastasia Ailamaki, Samuel Madden, Daniel Abadi, Gustavo Alonso, Sihem Amer-Yahia, Magdalena Balazinska, Philip A. Bernstein, Peter Boncz, Michael Cafarella, Surajit Chaudhuri, Susan Davidson, David DeWitt, Yanlei Diao, Xin Luna Dong, Michael Franklin, Juliana Freire, Johannes Gehrke, Alon Halevy, Joseph M. Hellerstein, Mark D. Hill, Stratos Idreos, Yannis Ioannidis, Christoph Koch, Donald Kossmann, Tim Kraska, Arun Kumar, Guoliang Li, Volker Markl, Renée Miller, C. Mohan, Thomas Neumann, Beng Chin Ooi, Fatma Ozcan, Aditya Parameswaran, Ippokratis Pandis, Jignesh M. Patel, Andrew Pavlo, Danica Porobic, Viktor Sanca, Michael Stonebraker, Julia Stoyanovich, Dan Suciu, Wang-Chiew Tan, Shiv Venkataraman, Matei Zaharia, and Stanley B. Zdonik

arXiv 2025

"LLMs ... make mistakes, effective explanation mechanisms ... will become increasingly important"

NL2SQL is a solved problem... Not!

Avrilia Floratou¹, Fotis Psallidas¹, Fuheng Zhao^{2,*}, Shaleen Deep¹, Gunther Hagleither³, Wangda Tan⁴, Joyce Cahoon¹, Rana Alotaibi¹, Jordan Henkel¹, Abhik Singla¹, Alex van Grootel¹, Brandon Chow¹, Kai Deng¹, Katherine Lin¹, Marcos Campos¹, Venkatesh Emani¹, Vivek Pandit¹, Victor Shnayder¹, Wenjing Wang¹, Carlo Curino¹

¹Microsoft, ² University of California, Santa Barbara, ^{3,4} Waii

CIDR 2024

"determine the intent of the user" .. "focusing on the alignment with the user's intention"

- 7 Shouldn't we make easier:
- for machines to write queries, and
 - for humans to understand queries?

We need a Rosetta Stone for relational queries:

1. a canonical representation of relational intent
("an abstract relational query language")
2. a refined vocabulary to separate intent from:
 - modalities (interface, and
 - conventions (semantic parameters)

The Composable Data Management System Manifesto

Pedro Pedreira
Meta Platforms Inc.
pedroerp@meta.com

Orri Erling
Meta Platforms Inc.
oerling@meta.com

Konstantinos
Karanasos
Meta Platforms Inc.
kkaranasos@meta.com

Scott Schneider
Meta Platforms Inc.
scottas@meta.com

Wes McKinney
Voltron Data
wes@voltrondata.com

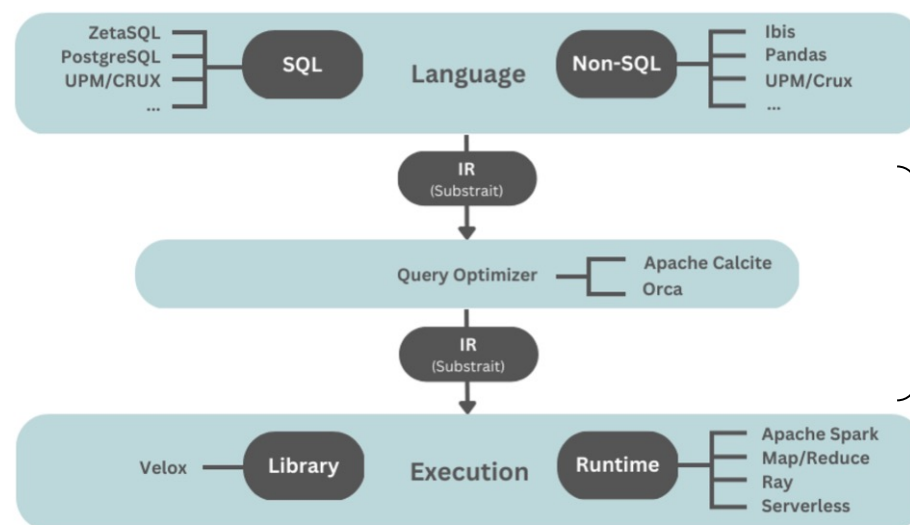
Satya R Valluri
Databricks Inc.
satya.valluri@databricks.com

Mohamed Zait
Databricks Inc.
mohamed.zait@databricks.com

Jacques Nadeau
Sundeck
jacques@sundeck.io

PVLDB 2023

"new modular data stack ... provides a stronger separation between language and execution ... the execution is language-independent"



User-level QLs
(e.g. SQL, SaneQL, Rel, ...)

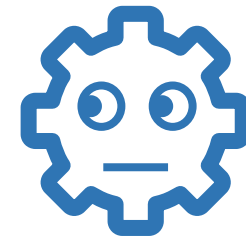
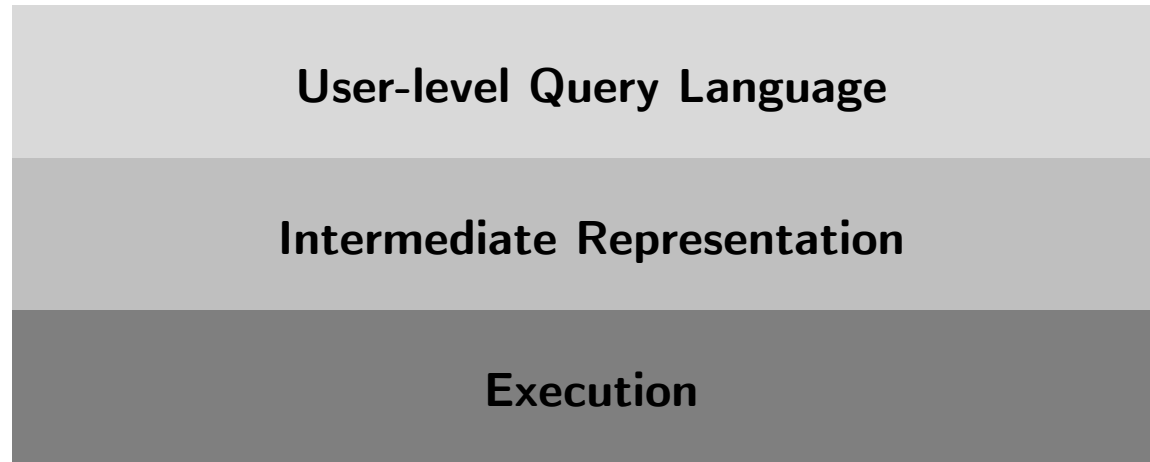
Intermediate Representations
(e.g. SDQL, Subtrait, ...)

Figure 1: Open source modular data stack outline.

An Abstract Relational Query Language

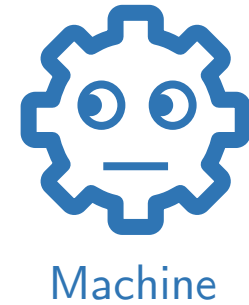
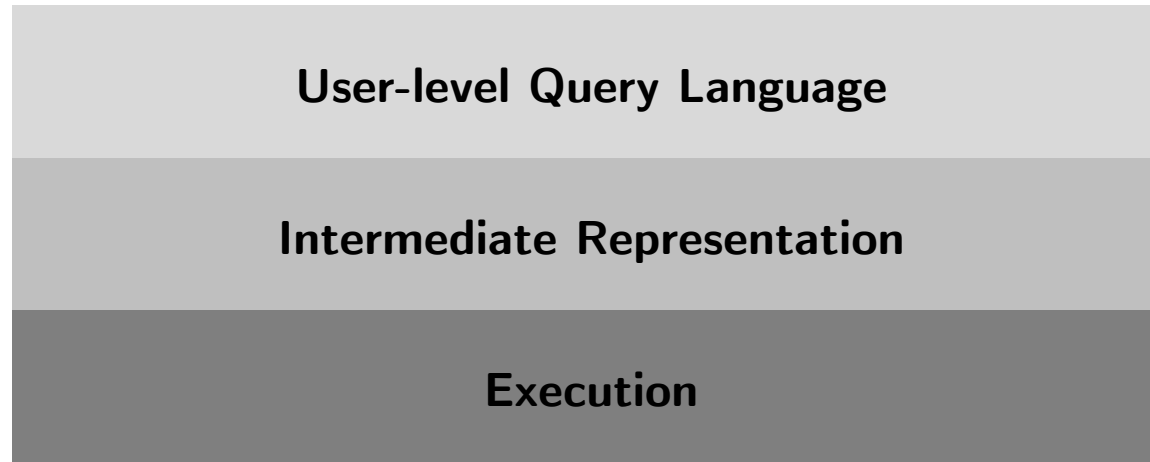


Human

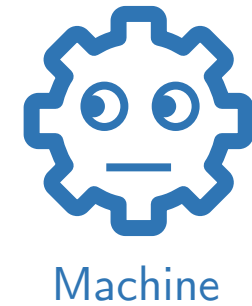
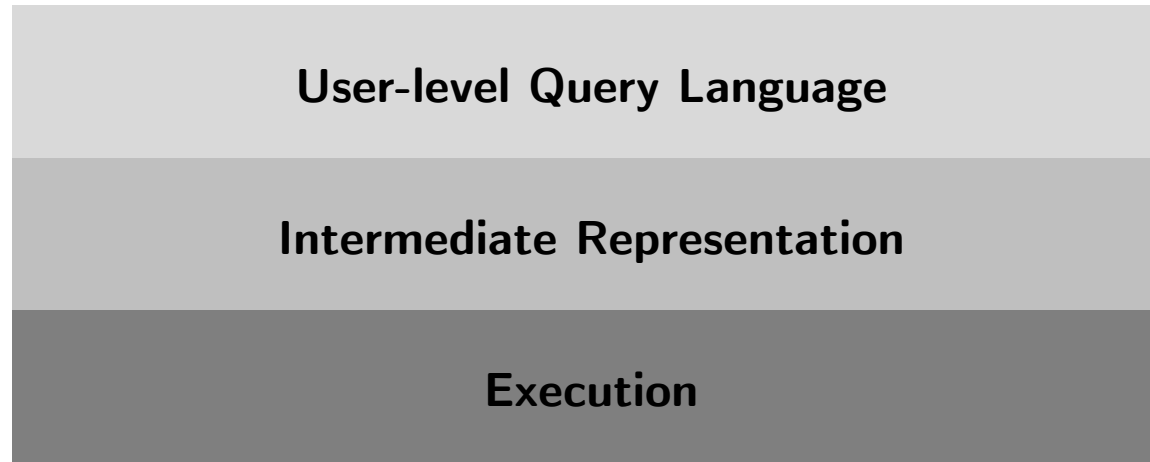


Machine

An Abstract Relational Query Language

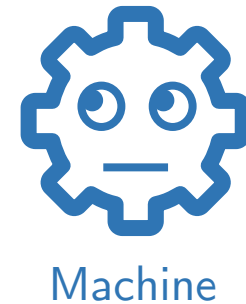
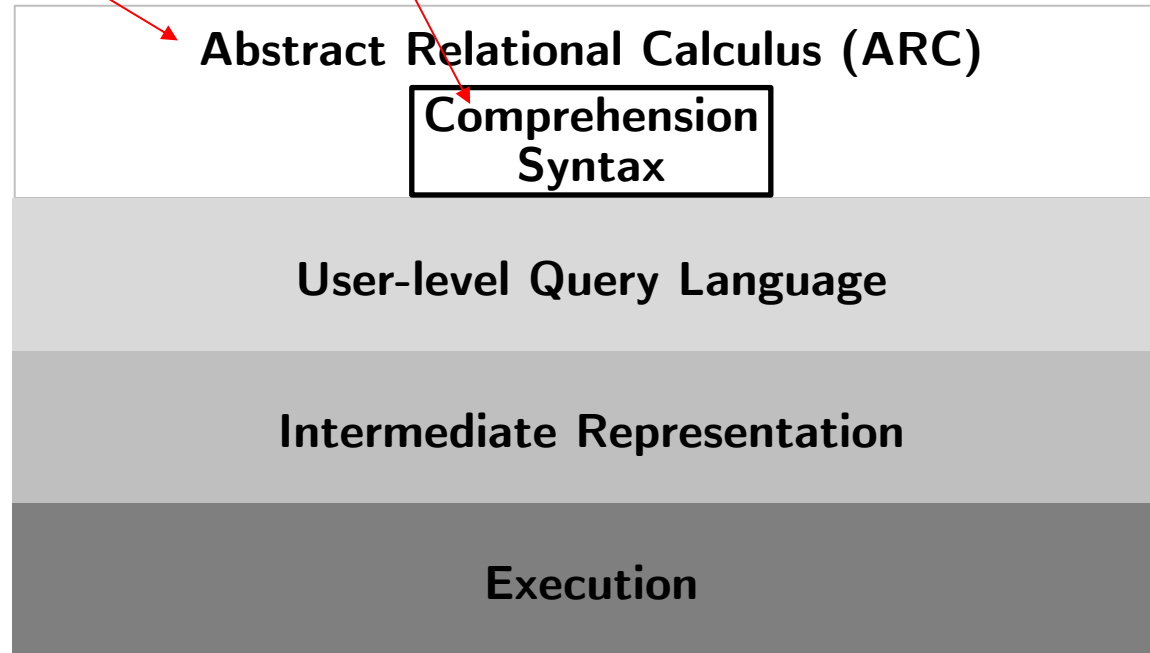


An Abstract Relational Query Language



An Abstract Relational Query Language

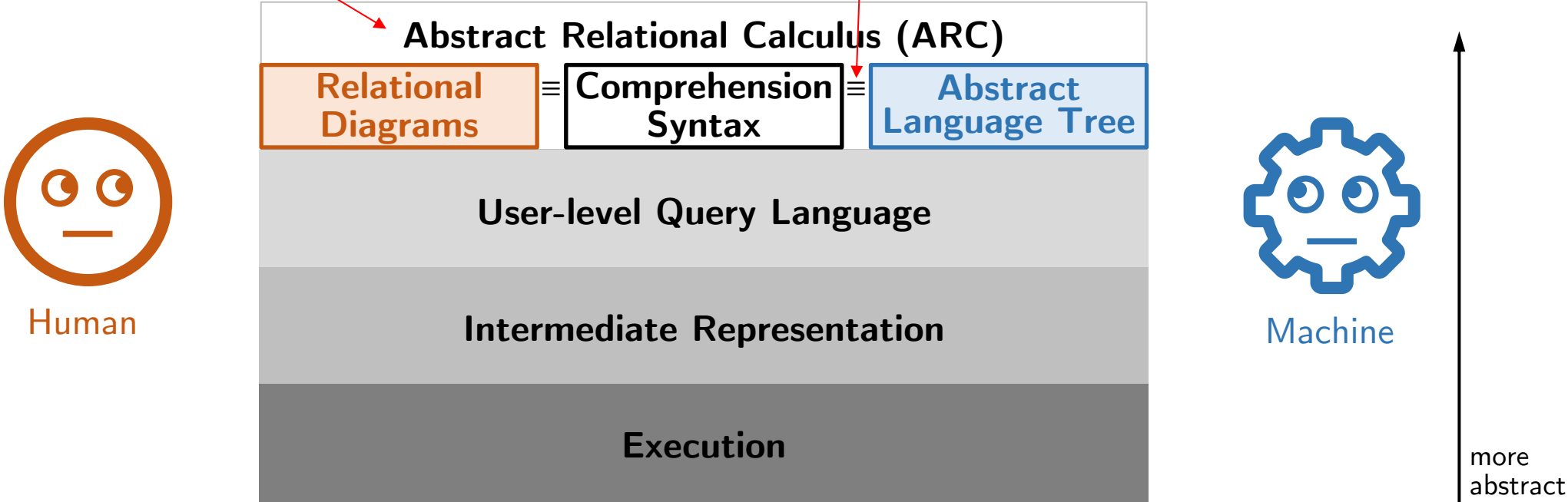
Reference language. Focus is on the intent: Separates "relational intent" from representation.
Textual representation



more
abstract

An Abstract Relational Query Language

Reference language. Focus is on the intent: Separates "relational intent" from representation.
Modality: alternative representation of the intent tailored to different audiences



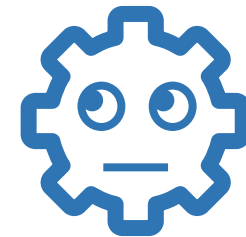
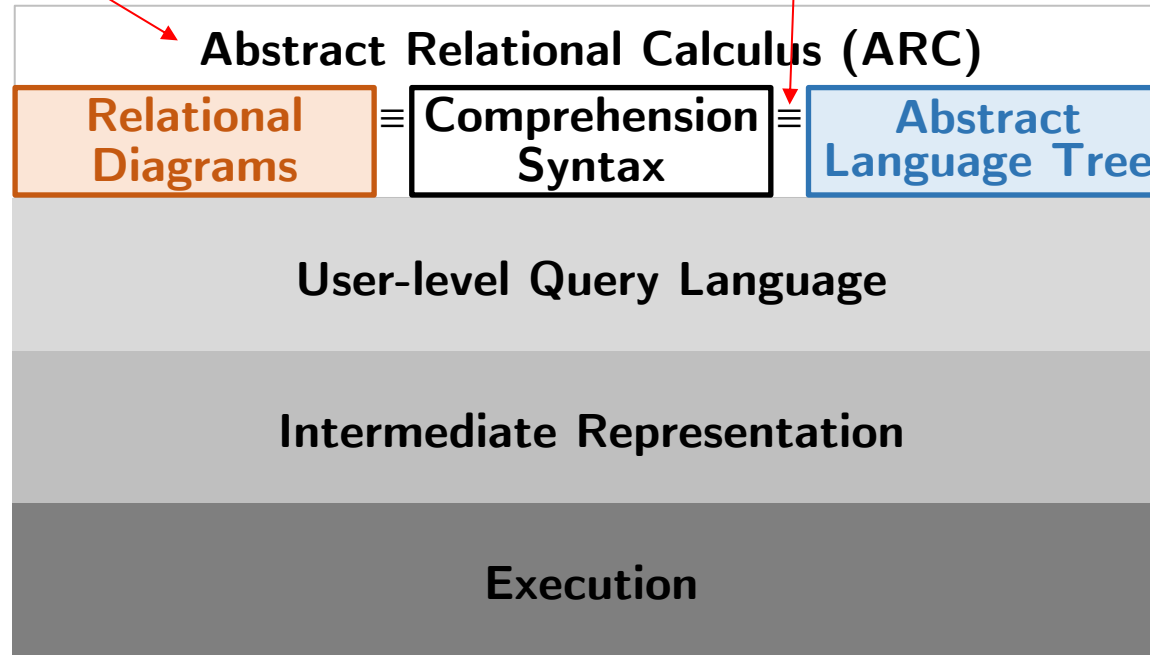
An Abstract Relational Query Language

Reference language. Focus is on the intent: Separates "relational intent" from representation.

Modality: alternative representation of the intent tailored to different audiences



Human



Machine

more
abstract

Convention: orthogonal environment-level semantic parameters under which the relational intent is interpreted (e.g., set vs. bag semantics, or treatment of null values)

An Abstract Relational Query Language

Reference language. Focus is on the intent: Separates "relational intent" from representation.

Modality: alternative representation of the intent tailored to different audiences

Our suggestion: **Abstract Relational Calculus (ARC)**:

- generalization of Tuple Relational Calculus (TRC) (grouping, aggregation, recursion, outer joins, IN with nulls, built-in predicates, external relations, ... etc.)
- allows split into 1 intent / 2 modality / 3 convention
- vocabulary for more fine-grained discussion, incl.:
- modularity through abstract relations

} see
paper

} talk

Convention: orthogonal environment-level semantic parameters under which the intent is interpreted (e.g., set vs. bag semantics, or treatment of null values)

Modalities & Conventions

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

SQL

```
select A as G  
from Left, Right  
where A = C
```

Datalog

$Q(x) :- \text{Left}(x, _), \text{Right}(x, _)$

Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [$
 $Q.G = l.A \wedge l.A = r.C]\}$

Languages differ in how
explicitly they encode and
surface the primitives of
relational languages

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

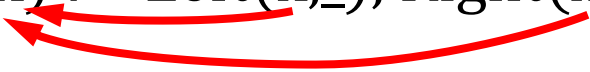
SQL

```
select A as G
from Left, Right
where A = C
```

assignment can be
from left or right
(logically equivalent)

Datalog

$Q(x) :- \text{Left}(x, _), \text{Right}(x, _)$



Languages differ in how
explicitly they encode and
surface the primitives of
relational languages

Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [$
 $Q.G = l.A \wedge l.A = r.C]\}$

assignment of the
output from Left

The tuple perspective is more explicit

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

SQL

```
select A as G
from Left, Right
where A = C
```

```
select A as G
from Left, Right
where A > C
```

assignment can be
from left or right
(logically equivalent)

Datalog

$Q(x) :- \text{Left}(x, _), \text{Right}(x, _)$

$Q(x) :- \text{Left}(x, _), \text{Right}(y, _), x > y$

normalized

Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [$
 $\boxed{Q.G = l.A} \wedge l.A = r.C]\}$

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [$
 $Q.G = l.A \wedge l.A > r.C]\}$

assignment of the
output from Left

The tuple perspective is more explicit

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

Languages

SQL

```
select A as G
from Left, Right
where A = C
```

modality: alternative representation of the intent tailored to different audiences

Datalog

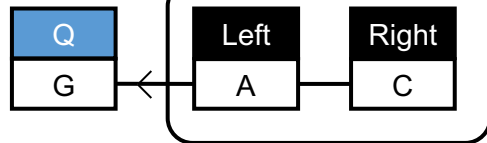
$Q(x) :- \text{Left}(x, _), \text{Right}(x, _)$

Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [Q.G = l.A \wedge l.A = r.C]\}$

COLLECTION

```
├ HEAD: Q(G)
├ QUANTIFIER ∃
├   ├── BINDING: l ∈ Left
├   ├── BINDING: r ∈ Right
├   └ AND ∧
├       ├── PREDICATE: Q.G = l.A
├       └ PREDICATE: l.A = r.C
```



Relational Diagram

Set comprehension based textual notation

Abstract Language Tree

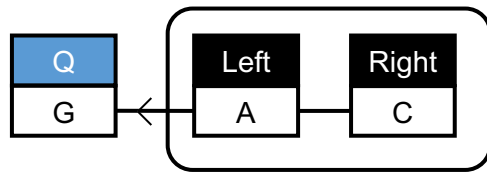
Modalities

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

modality: alternative representation of the intent tailored to different audiences

one-to-one correspondence
b/w relational concepts
across the modalities



Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [$
 $Q.G = l.A \wedge l.A = r.C]\}$

```
COLLECTION
├ HEAD: Q(G)
├ QUANTIFIER ∃
│   ├── BINDING: l ∈ Left
│   ├── BINDING: r ∈ Right
│   └ AND ∧
│       ├── PREDICATE: Q.G = l.A
│       └ PREDICATE: l.A = r.C
```

Relational Diagram

Set comprehension based textual notation

Abstract Language Tree

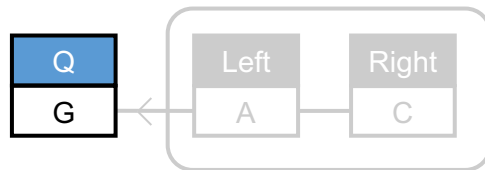
Modalities

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

modality: alternative representation of the intent tailored to different audiences

one-to-one correspondence b/w relational concepts across the modalities



Relational Diagram

Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [Q.G = l.A \wedge l.A = r.C]\}$

Set comprehension based textual notation

```
COLLECTION
├ HEAD: Q(G)
├ QUANTIFIER ∃
│   ├── BINDING: l ∈ Left
│   ├── BINDING: r ∈ Right
│   └ AND ∧
│       ├── PREDICATE: Q.G = l.A
│       └ PREDICATE: l.A = r.C
```

Abstract Language Tree

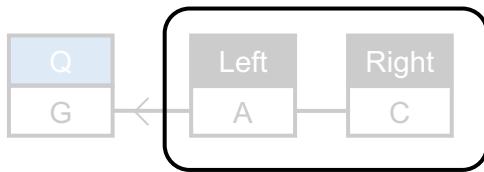
Modalities

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

modality: alternative representation of the intent tailored to different audiences

one-to-one correspondence
b/w relational concepts
across the modalities



Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [$
 $Q.G = l.A \wedge l.A = r.C]\}$

```
COLLECTION
├ HEAD: Q(G)
├ QUANTIFIER  $\exists$ 
│   ├── BINDING:  $l \in \text{Left}$ 
│   ├── BINDING:  $r \in \text{Right}$ 
│   └ AND  $\wedge$ 
│       ├── PREDICATE:  $Q.G = l.A$ 
│       └ PREDICATE:  $l.A = r.C$ 
```

Relational Diagram

Set comprehension based textual notation

Abstract Language Tree

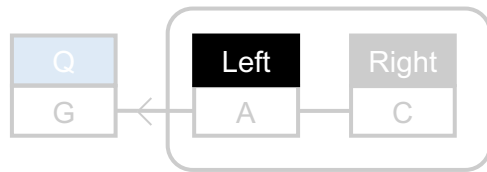
Modalities

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

modality: alternative representation of the intent tailored to different audiences

one-to-one correspondence b/w relational concepts across the modalities



Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [Q.G = l.A \wedge l.A = r.C]\}$

```
COLLECTION
├ HEAD: Q(G)
├ QUANTIFIER ∃
│   ├── BINDING: l ∈ Left
│   ├── BINDING: r ∈ Right
│   └ AND ∧
│       ├── PREDICATE: Q.G = l.A
│       └ PREDICATE: l.A = r.C
```

Relational Diagram

Set comprehension based textual notation

Abstract Language Tree

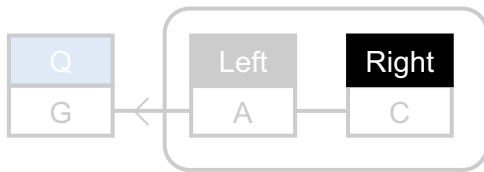
Modalities

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

modality: alternative representation of the intent tailored to different audiences

one-to-one correspondence
b/w relational concepts
across the modalities



Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [$
 $Q.G = l.A \wedge l.A = r.C]\}$

```
COLLECTION
├ HEAD: Q(G)
├ QUANTIFIER ∃
│   ├── BINDING: l ∈ Left
│   ├── BINDING: r ∈ Right
│   └ AND ∧
│       ├── PREDICATE: Q.G = l.A
│       └ PREDICATE: l.A = r.C
```

Relational Diagram

Set comprehension based textual notation

Abstract Language Tree

Modalities

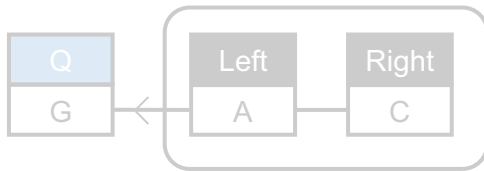
Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

modality: alternative representation of the intent tailored to different audiences

one-to-one correspondence
b/w relational concepts
across the modalities

conjunction = juxtaposition



Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [$
 $Q.G = l.A \wedge l.A = r.C]\}$

```
COLLECTION
├ HEAD: Q(G)
├ QUANTIFIER ∃
│   ├── BINDING: l ∈ Left
│   ├── BINDING: r ∈ Right
│   └ AND ∧
│       ├── PREDICATE: Q.G = l.A
│       └ PREDICATE: l.A = r.C
```

Relational Diagram

Set comprehension based textual notation

Abstract Language Tree

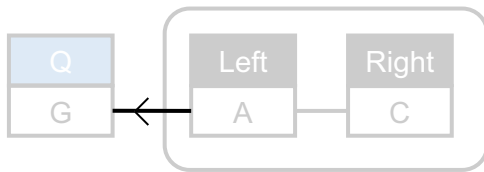
Modalities

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

modality: alternative representation of the intent tailored to different audiences

one-to-one correspondence b/w relational concepts across the modalities



Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [Q.G = l.A \wedge l.A = r.C]\}$

```
COLLECTION
├ HEAD: Q(G)
├ QUANTIFIER ∃
│   ├── BINDING: l ∈ Left
│   ├── BINDING: r ∈ Right
│   └ AND ∧
│       ├── PREDICATE: Q.G = l.A
│       └ PREDICATE: l.A = r.C
```

Relational Diagram

Set comprehension based textual notation

Abstract Language Tree

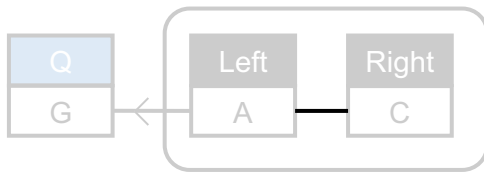
Modalities

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

modality: alternative representation of the intent tailored to different audiences

one-to-one correspondence b/w relational concepts across the modalities



Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [Q.G = l.A \wedge l.A = r.C]\}$

```
COLLECTION
├ HEAD: Q(G)
├ QUANTIFIER ∃
│   ├── BINDING: l ∈ Left
│   ├── BINDING: r ∈ Right
│   └ AND ∧
│       ├── PREDICATE: Q.G = l.A
│       └ PREDICATE: l.A = r.C
```

Relational Diagram

Set comprehension based textual notation

Abstract Language Tree

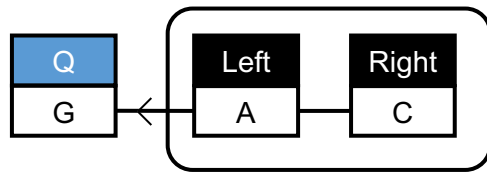
Modalities

Languages vs. Modalities

Relational schema: Left(A,B), Right(C,D)

modality: alternative representation of the intent tailored to different audiences

one-to-one correspondence
b/w relational concepts
across the modalities



Tuple Relational Calculus (TRC)

$\{Q(G) \mid \exists l \in \text{Left}, r \in \text{Right} [$
 $Q.G = l.A \wedge l.A = r.C]\}$

```
COLLECTION
├ HEAD: Q(G)
├ QUANTIFIER ∃
│   ├── BINDING: l ∈ Left
│   ├── BINDING: r ∈ Right
│   └ AND ∧
│       ├── PREDICATE: Q.G = l.A
│       └ PREDICATE: l.A = r.C
```

Relational Diagram

Set comprehension based textual notation

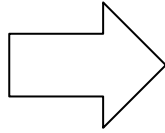
Abstract Language Tree

Modalities

Conventions

SQL

```
select A,  
  (select sum(D) as sm  
   from Right  
   where A = C)  
from Left
```

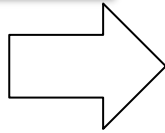


A	sm
a	3

Left		Right	
<u>A</u>	B	C	D
a	1	a	1
		a	2

Datalog (Soufflé)

```
Q(x, sum y: {Right(x, y)}) :- Left(x, _).
```

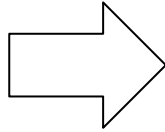


A	sm
a	3

Conventions

SQL

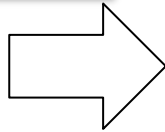
```
select A,  
  (select sum(D) as sm  
   from Right  
   where A = C)  
from Left
```



A	sm
a	3
b	7

Datalog (Soufflé)

```
Q(x, sum y: {Right(x, y)}) :- Left(x, _).
```



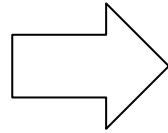
A	sm
a	3
b	7

Left		Right	
<u>A</u>	B	C	D
a	1	a	1
b	2	a	2
		b	3
		b	4

Conventions

SQL

```
select A,  
  (select sum(D) as sm  
   from Right  
   where A = C)  
from Left
```

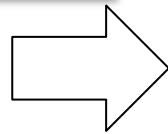


A	sm
a	3
b	7
c	null

Left		Right	
<u>A</u>	B	C	D
a	1	a	1
b	2	a	2
c	3	b	3
		b	4

Datalog (Soufflé)

```
Q(x, sum y: {Right(x, y)}) :- Left(x, _).
```



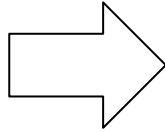
A	sm
a	3
b	7
c	0



Conventions

SQL

```
select A,  
  (select sum(D) as sm  
   from Right  
   where A = C)  
from Left
```



A	sm
a	3
b	7
c	null

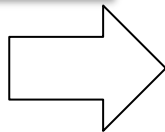


Queries

Left		Right	
A	B	C	D
a	1	a	1
b	2	a	2
c	3	b	3
		b	4

Datalog (Soufflé)

```
Q(x, sum y: {Right(x, y)}) :- Left(x, _).
```



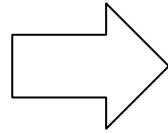
A	sm
a	3
b	7
c	0



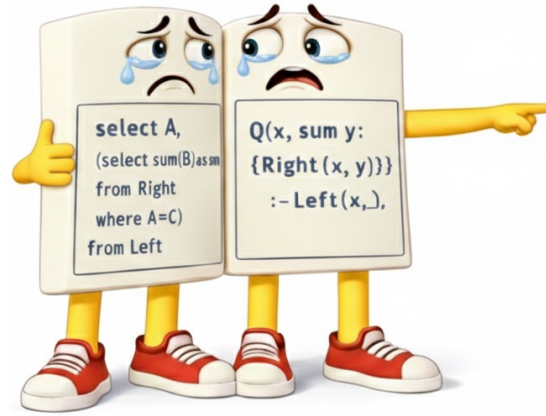
Conventions

SQL

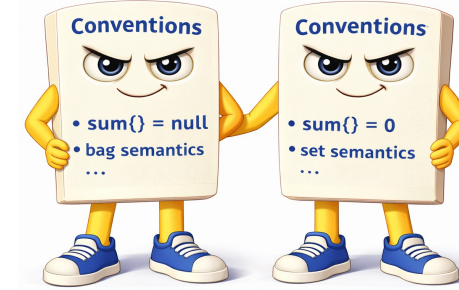
```
select A,  
  (select sum(D) as sm  
   from Right  
  where A = C)  
from Left
```



A	sm
a	3
b	7
c	null



Queries

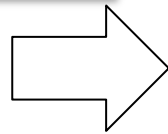


Conventions

Left		Right	
A	B	C	D
a	1	a	1
b	2	a	2
c	3	b	3
		b	4

Datalog (Soufflé)

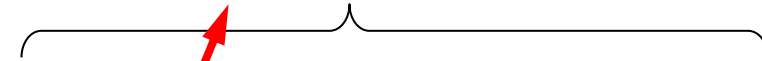
```
Q(x, sum y: {Right(x, y)}) :- Left(x, _).
```



A	sm
a	3
b	7
c	0



Language



Query



orthogonal environment-level
semantic parameters under
which the query is interpreted

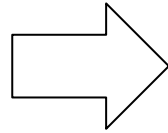
Convention

swappable

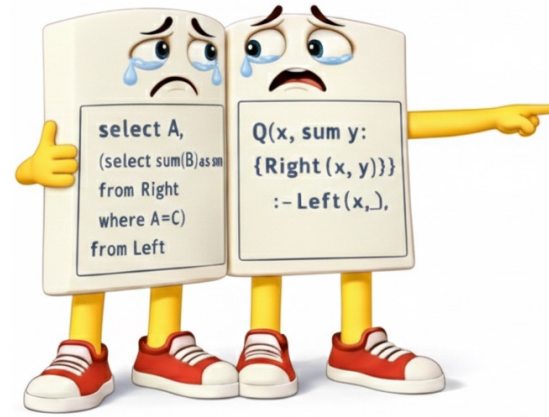
Conventions

SQL

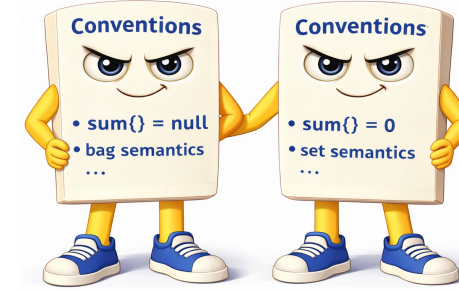
```
select A,
  (select sum(D) as sm
   from Right
   where A = C)
from Left
```



A	sm
a	3
b	7
c	null



Queries

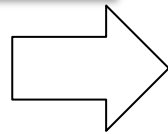


Conventions

Left		Right	
A	B	C	D
a	1	a	1
b	2	a	2
c	3	b	3
		b	4

Datalog (Soufflé)

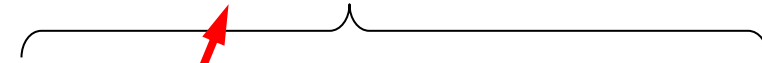
```
Q(x, sum y: {Right(x, y)}) :- Left(x, _).
```



A	sm
a	3
b	7
c	0



Language



Query



3. Convention

orthogonal environment-level semantic parameters under which the query is interpreted

swappable

1. Relational Intent

the relational entities & the compositional structure that determine intent

2. Modality

alternative representations of that intent tailored to different audiences

$(\text{Intent} \times \text{Modality} = \text{Query}) \times \text{Conventions} = \text{Semantics}$

orthogonal environment-level parameters
under which the query is interpreted

Queries
(in various syntax)

```
select A,  
  (select sum(D) as sm  
   from Right  
   where A = C)  
from Left
```

Conventions

Conventions

- $\text{sum}(\{\}) = 0$
- **set** semantics
- ...

*composable / modular
through independence,
separation of concerns,
= factorization*

```
Q(x, sum y: {Right(x, y)}) :- Left(x, _).
```

Conventions

- $\text{sum}(\{\}) = \text{null}$
- **bag** semantics
- ...

$(\text{Intent} \times \text{Modality} = \text{Query}) \times \text{Conventions} = \text{Semantics}$

Queries
(in various syntax)

```
select A,  
  (select sum(D) as sm  
   from Right  
   where A = C)  
from Left
```

```
Q(x, sum y: {Right(x, y)}) :- Left(x, _).
```

orthogonal environment-level parameters
under which the query is interpreted

Collection
Convention

Convention
#539
• set semantics

Null
Convention

Convention
#1483
• $\text{sum}(\{\}) = 0$

composable / modular
through independence,
separation of concerns,
= factorization

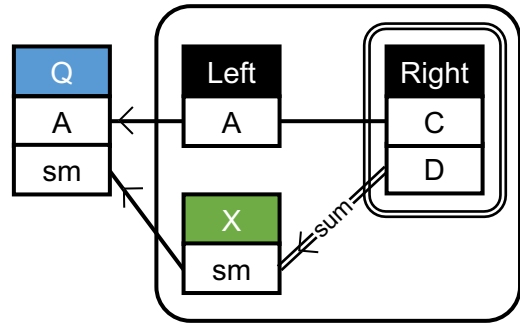
Convention
#538
• bag semantics

Convention
#1484
• $\text{sum}(\{\}) = \text{null}$

(Intent × Modality = Query) × Conventions = Semantics

the relational entities &
the compositional structure
that determine intent

Relational Intent



Relational Diagram

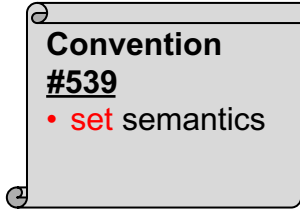
Queries (in various syntax)

```
select A,  
  (select sum(D) as sm  
   from Right  
  where A = C)  
from Left
```

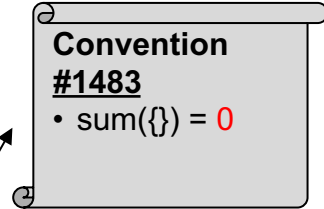
```
Q(x, sum y: {Right(x, y)}) :- Left(x, _).
```

orthogonal environment-level parameters
under which the query is interpreted

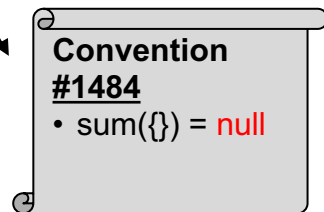
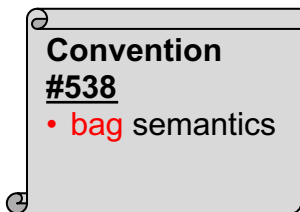
Collection Convention



Null Convention



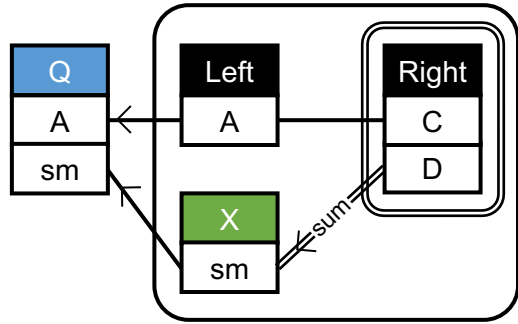
composable / modular
through independence,
separation of concerns,
= factorization



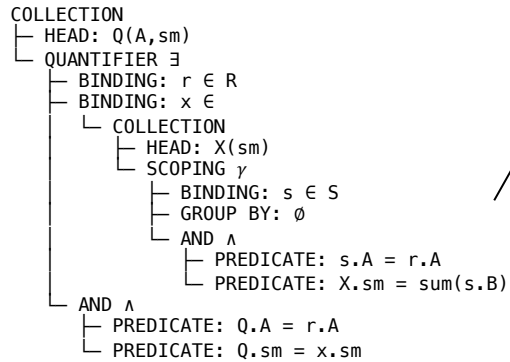
(Intent × Modality = Query) × Conventions = Semantics

the relational entities &
the compositional structure
that determine intent

Relational Intent × Modality



Relational Diagram



Abstract Language Tree

Modality

alternative representations
of that intent tailored to
different audiences

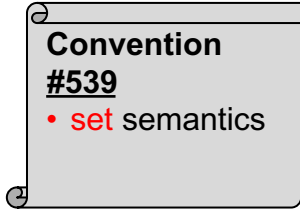
Queries
(in various syntax)

```
select A,  
  (select sum(D) as sm  
   from Right  
   where A = C)  
from Left
```

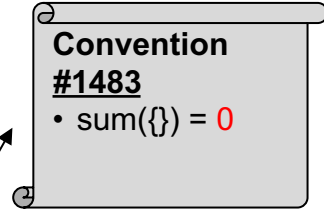
```
Q(x, sum y: {Right(x, y)}) :- Left(x, _).
```

orthogonal environment-level parameters
under which the query is interpreted

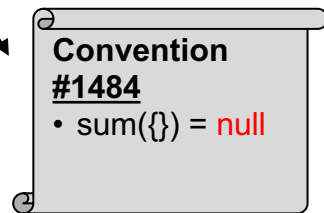
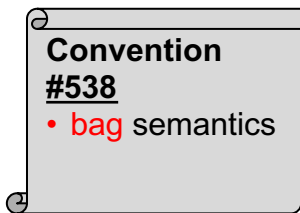
**Collection
Convention**



**Null
Convention**



composable / modular
through independence,
separation of concerns,
= factorization



Two debates that we analyze today:

(1) Is linear dataflow inherently :
"easier to read" than inside-out?

(2) Are dataflow languages compositional, and inside-out is not?

Inside-out (nesting/single-block) vs. pipelined dataflow

Claim: Pipelines are a lot more intuitive than SQL's "inside-out" dataflow.

Piped dataflow $\approx$ linear sequence of operators. Examples:

- DFQL ("*Dataflow QL*") [Clark,Wu'94] ("*ease-of-use*")
- PigLatin [Olston+'08] (*dataflow language, step-by-step where each step carries out a single data transformation, easier for programmers, explicit intermediate results*)
- Dataframes, such as Pandas/Python, or dplyr/R (*operator chaining*)
- FunSQL.jl
- PRQL ("*Pipelined Relational QL*" = "Prequel")
- Malloy
- SaneQL [Neumann, Leis'24]
- Google's SQL Pipe Syntax [Shute+'24]

[Olston+'08] Pig Latin: A Not-So-Foreign Language for Data Processing, SIGMOD 2008. <https://doi.org/10.1145/1376616.1376726> [PRQL'22]: a simple, powerful, pipelined SQL replacement. <https://prql-lang.org/>
[Malloy'22]: A modern open source language for analyzing, transforming, and modeling data. <https://www.malloydata.dev/> [FunSQL'21]: a Julia library for compositional construction of SQL queries. <https://github.com/MechanicalRabbit/FunSQL.jl> [Neumann, Leis'24] A Critique of Modern SQL And A Proposal Towards A Simple and Expressive Query Language, CIDR 2024, <https://mail.vldb.org/cidrcdb/papers/2024/p48-neumann.pdf> [Shute+'24] SQL Has Problems. We Can Fix Them: Pipe Syntax In SQL, PVLDB. <https://vldb.org/pvldb/vol17/p4051-shute.pdf>

Inside-out (nesting/single-block) vs. pipelined dataflow

Claim: Pipelines are a lot more intuitive than SQL's "inside-out" dataflow.

Piped dataflow $\approx$ linear sequence of operators. Examples:

- DFQL ("*Dataflow QL*") [Clark,Wu'94] ("*ease-of-use*")
- PigLatin [Olston+'08] (*dataflow language, step-by-step where each step carries out a single data transformation, easier for programmers, explicit intermediate results*)
- Dataframes, such as Pandas/Python, or dplyr/R (*operator chaining*)
- FunSQL.jl
- PRQL ("*Pipelined Relational QL*" = "Prequel")
- Malloy
- **SaneQL** [Neumann, Leis'24]
- **Google's SQL pipe syntax** [Shute+'24]

*allows parameterized functions
(thus not strict "pipeline")*

(extend instead of replace SQL)

[Olston+'08] Pig Latin: A Not-So-Foreign Language for Data Processing, SIGMOD 2008. <https://doi.org/10.1145/1376616.1376726> [PRQL'22]: a simple, powerful, pipelined SQL replacement. <https://prql-lang.org/>
[Malloy'22]: A modern open source language for analyzing, transforming, and modeling data. <https://www.malloydata.dev/> [FunSQL'21]: a Julia library for compositional construction of SQL queries. <https://github.com/MechanicalRabbit/FunSQL.jl> [Neumann, Leis'24] A Critique of Modern SQL And A Proposal Towards A Simple and Expressive Query Language, CIDR 2024, <https://mail.vldb.org/cidrdb/papers/2024/p48-neumann.pdf> [Shute+'24] SQL Has Problems. We Can Fix Them: Pipe Syntax In SQL, PVLDB. <https://vldb.org/pvldb/vol17/p4051-shute.pdf>

Simplified TPC-H13 (customer distribution) : Inside-out vs. pipelines

SQL

```
select custct, count(*) as custdist
from
  (select C.custkey, count(O.orderkey) as custct
   from customer C
   left outer join orders O
   on C.custkey=O.custkey
   and O.comment not like '%unusual%'
   group by C.custkey) as X
group by custct
```

You do **not** need to understand the queries to follow the argument!
Just follow the flow!

Claim: A piped linear syntax is easier-to-understand than "inside-out"

Pipe syntax

```
from customer as C
|> left outer join orders as O
  on C.custkey = O.custkey
  and O.comment not like '%unusual%'
|> aggregate count(O.orderkey) as custct
  group by C.custkey
|> aggregate count(*) as custdist
  group by custct
```

Shouldn't grouping come before aggregation, as in PigLatin?

"A piped linear syntax is easier to understand than inside-out"

SQL

```
select custct, count(*) as custdist
from
  (select C.custkey, count(O.orderkey) as custct
   from customer C
   left outer join orders O
   on C.custkey=O.custkey
   and O.comment not like '%unusual%'
   group by C.custkey) as X
group by custct
```

Claim: A piped linear syntax is easier-to-understand than "inside-out"

Pipe syntax

```
from customer as C
|> left outer join orders as O
  on C.custkey = O.custkey
  and O.comment not like '%unusual%'
|> aggregate count(O.orderkey) as custct
  group by C.custkey
|> aggregate count(*) as custdist
  group by custct
```

"A piped linear syntax is easier to understand than inside-out"

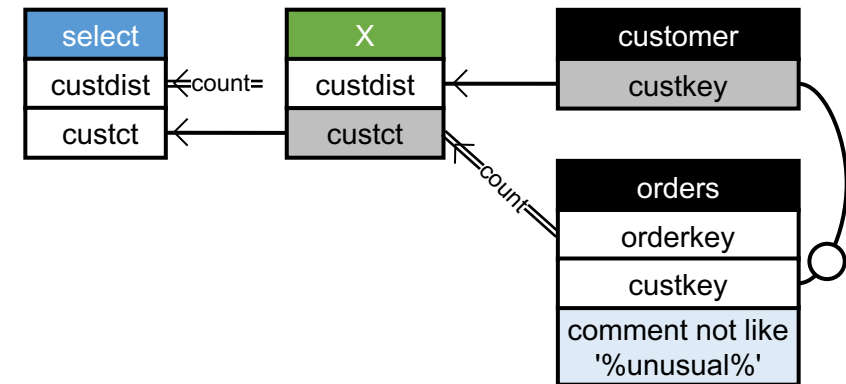
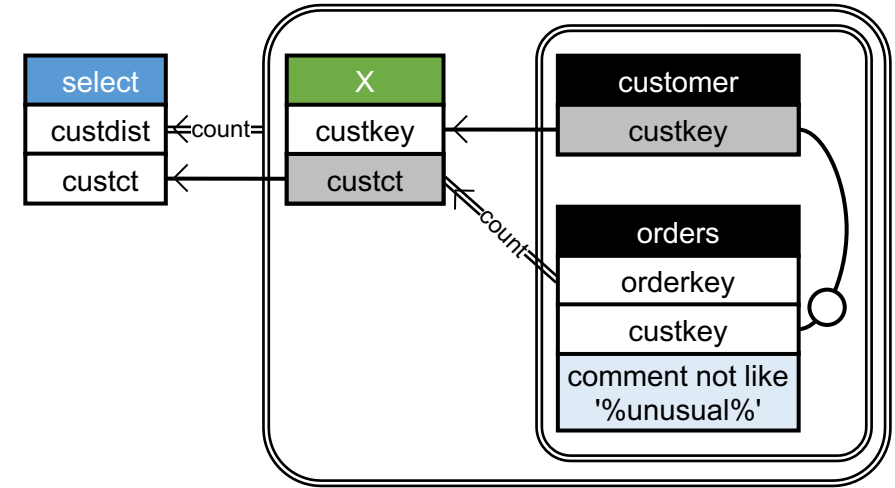
SQL

```
select custct, count(*) as custdist
from
  (select C.custkey, count(O.orderkey) as custct
   from customer C
   left outer join orders O
   on C.custkey=O.custkey
   and O.comment not like '%unusual%'
   group by C.custkey) as X
group by custct
```

Claim: A piped linear syntax is easier-to-understand than "inside-out"

Pipe syntax

```
from customer as C
|> left outer join orders as O
  on C.custkey = O.custkey
  and O.comment not like '%unusual%'
|> aggregate count(O.orderkey) as custct
  group by C.custkey
|> aggregate count(*) as custdist
  group by custct
```



"A piped linear syntax is easier to understand than inside-out"

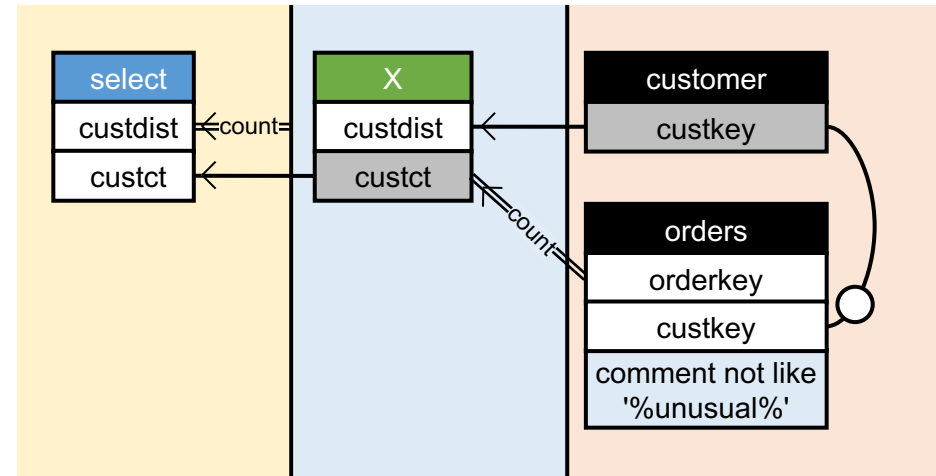
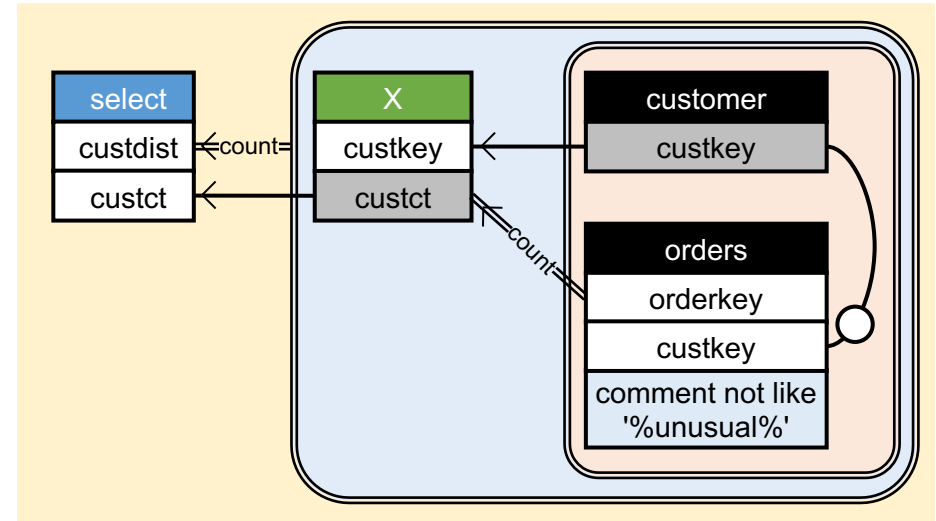
SQL

```
select custct, count(*) as custdist
from
  (select C.custkey, count(O.orderkey) as custct
   from customer C
   left outer join orders O
   on C.custkey=O.custkey
   and O.comment not like '%unusual%'
   group by C.custkey) as X
group by custct
```

Claim: A piped linear syntax is easier-to-understand then "inside-out"

Pipe syntax

```
from customer as C
|> left outer join orders as O
  on C.custkey = O.custkey
  and O.comment not like '%unusual%'
|> aggregate count(O.orderkey) as custct
  group by C.custkey
|> aggregate count(*) as custdist
  group by custct
```



"A piped linear syntax is easier to understand than inside-out"

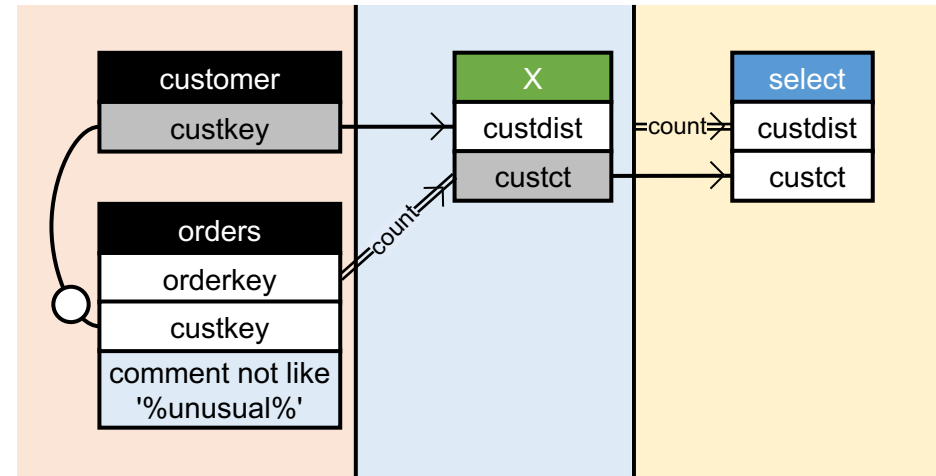
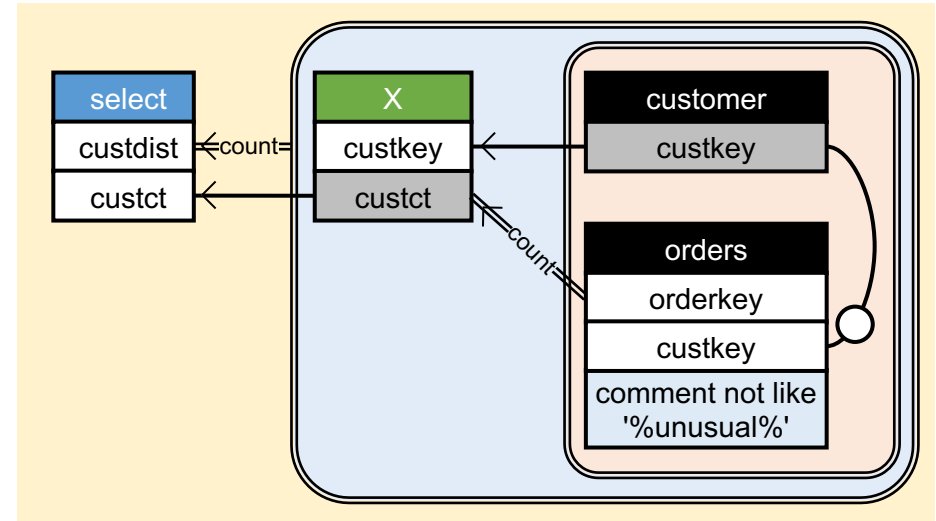
SQL

```
select custct, count(*) as custdist
from
  (select C.custkey, count(O.orderkey) as custct
   from customer C
   left outer join orders O
   on C.custkey=O.custkey
   and O.comment not like '%unusual%'
   group by C.custkey) as X
group by custct
```

Claim: A piped linear syntax is easier-to-understand than "inside-out"

Pipe syntax

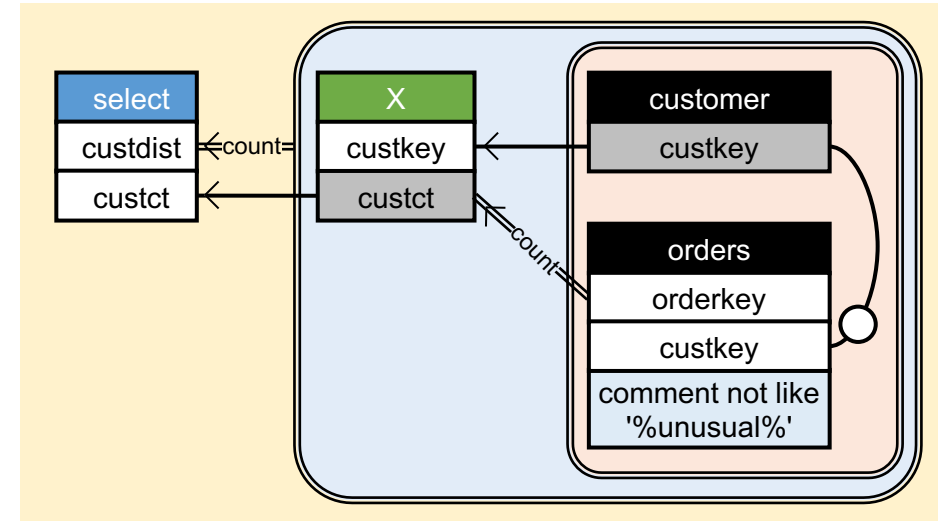
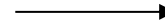
```
from customer as C
|> left outer join orders as O
  on C.custkey = O.custkey
  and O.comment not like '%unusual%'
|> aggregate count(O.orderkey) as custct
  group by C.custkey
|> aggregate count(*) as custdist
  group by custct
```



"A piped linear syntax is easier to understand than inside-out"

SQL

```
select custct, count(*) as custdist
from
  (select C.custkey, count(0.orderkey) as custct
   from customer C
   left outer join orders 0
   on C.custkey=0.custkey
   and 0.comment not like '%unusual%'
   group by C.custkey) as X
group by custct
```



Claim: A piped linear syntax is easier-to-understand than "inside-out"

Pipe syntax

```
from customer as C
|> left outer join orders as 0
  on C.custkey = 0.custkey
  and 0.comment not like '%unusual%'
|> aggregate count(0.orderkey) as custct
  group by C.custkey
|> aggregate count(*) as custdist
  group by custct
```

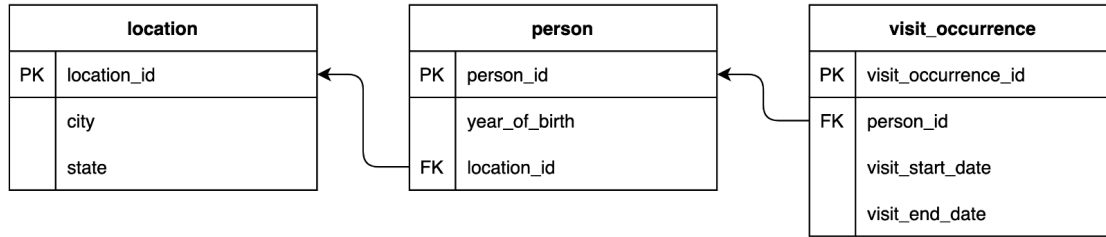
Counter-claim:

- easy-to-understand depends on "modality", not piped linear syntax vs. "inside-out"

1. Pipes (in text) reduce "split-attention"
2. but they also need nestings and can't avoid "split attention" (partial orders / DAGs)

Linearizing partial orders also leads to nesting

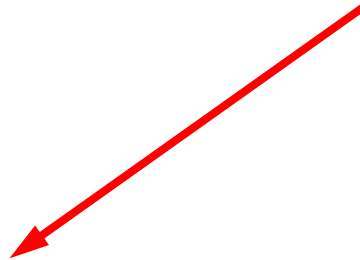
Q: When was the last time each person born between 1930 and 1940 and living in Illinois was seen by a healthcare provider?



You do **not** need to understand the queries to follow the argument! Just follow the flow!

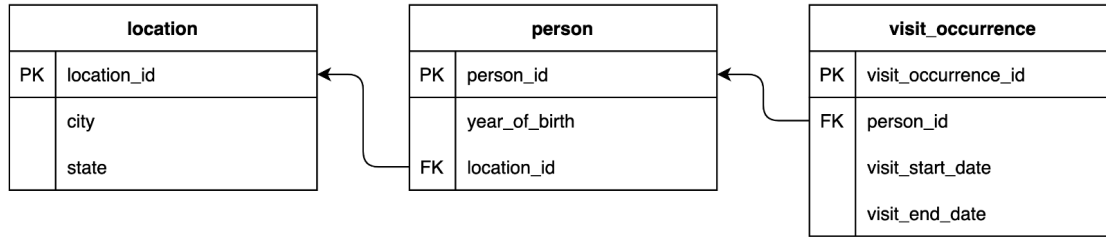
(I also have the full SQL query larger in a backup slide)

"FunSQL"

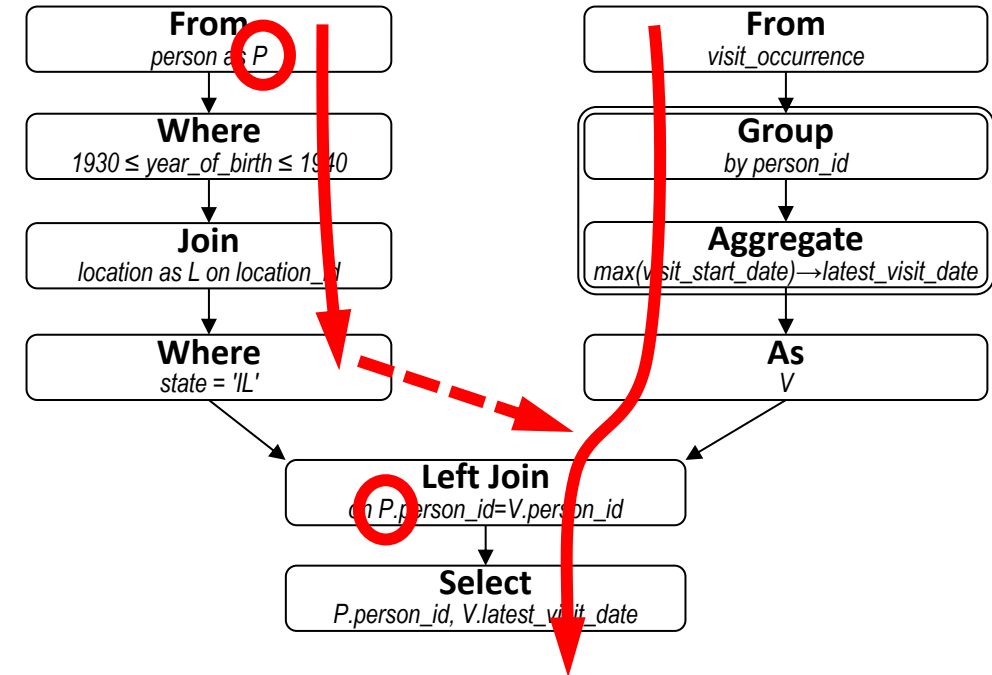


Linearizing partial orders also leads to nesting

Q: When was the last time each person born between 1930 and 1940 and living in Illinois was seen by a healthcare provider?



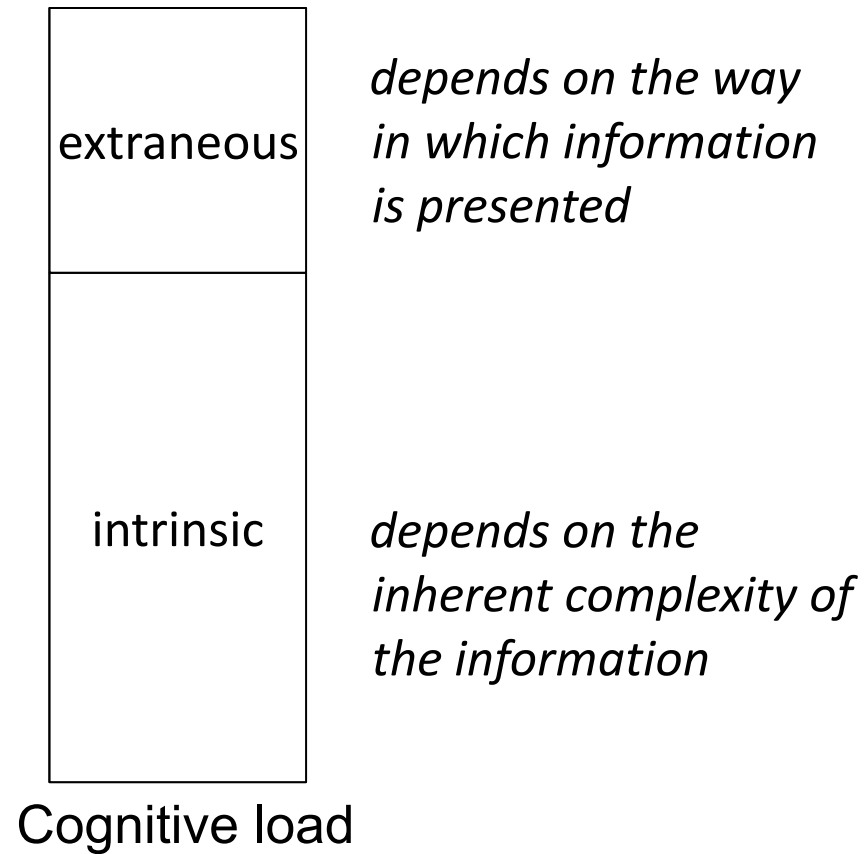
```
from person as P
|> where year_of_birth >= 1930
    and year_of_birth <= 1940
|> join location as L
    on P.location_id = L.location_id
|> where L.state = 'IL'
|> left join (
    from visit_occurrence
    |> aggregate max(visit_start_date)
        as latest_visit_date
    group by person_id) as V
    on P.person_id = V.person_id
|> select P.person_id, V.latest_visit_date;
```



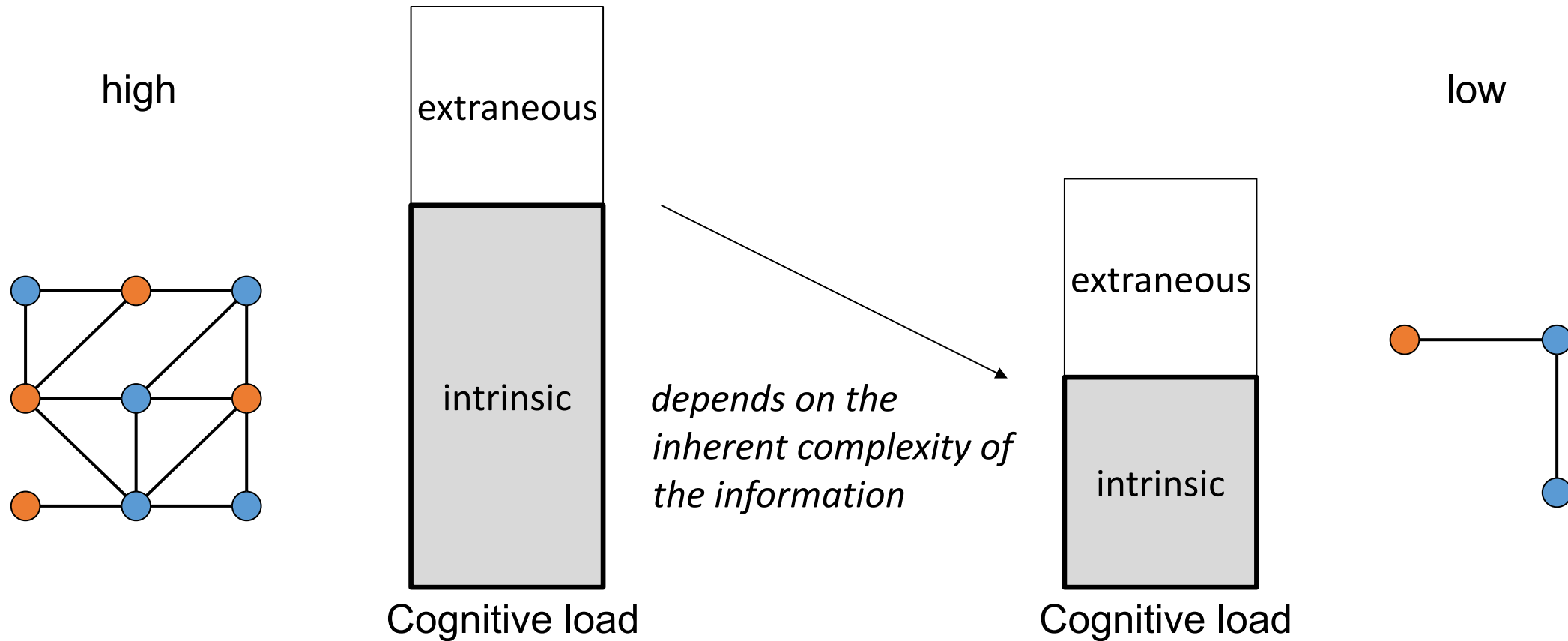
Serialization of a partial order requires jumping around via some reference (define and re-use later): this cannot be avoided!

Cognitive load theory: "split-attention" when two related pieces of information are spatially separated

Cognitive load: intrinsic vs extraneous



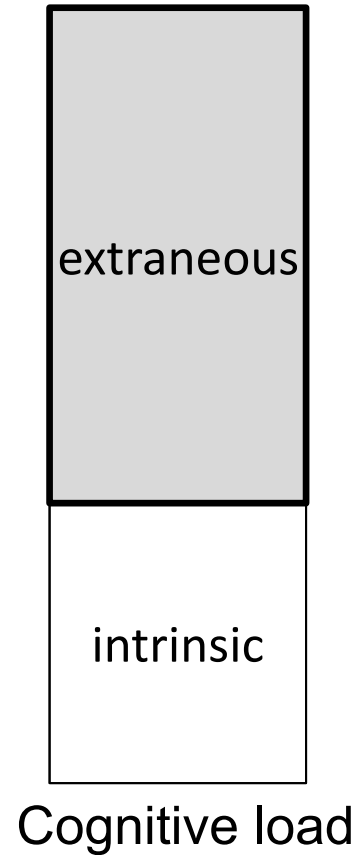
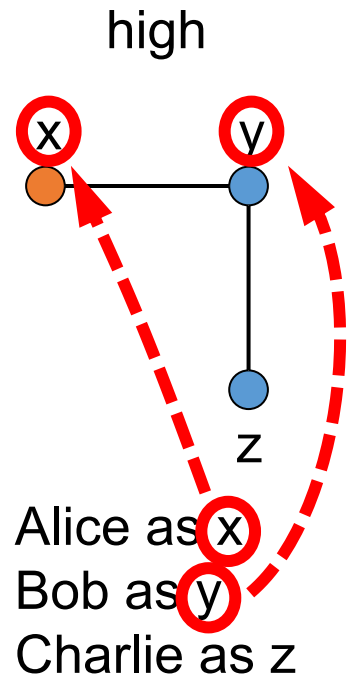
Cognitive load: intrinsic vs extraneous



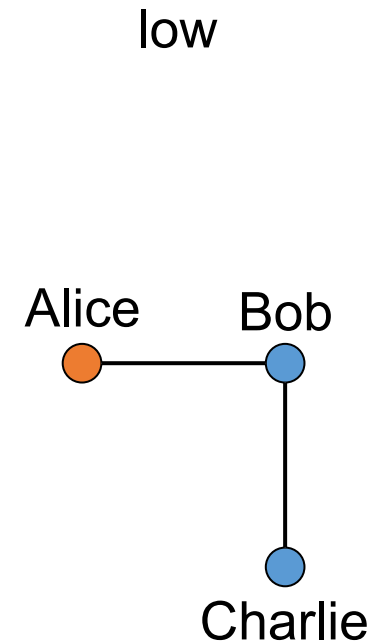
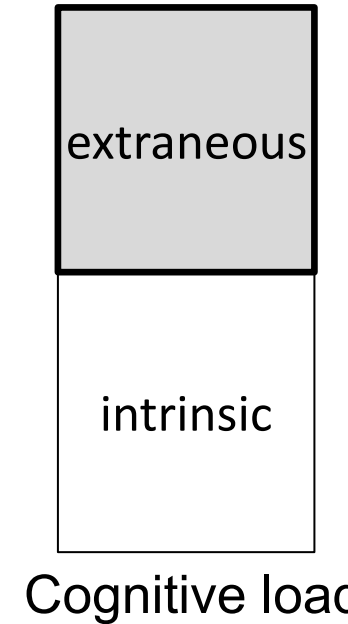
Cognitive load: intrinsic vs extraneous

split-attention: when attention is split between spatially separated pieces of information

spatial contiguity: when related information is near one another



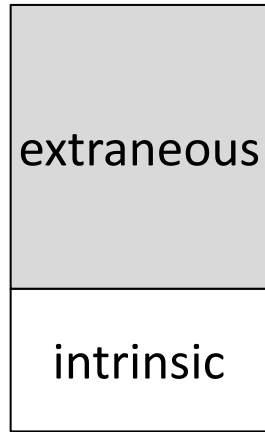
*depends on the way
in which information
is presented*



Pipelines reduce split-points, diagrams remove them

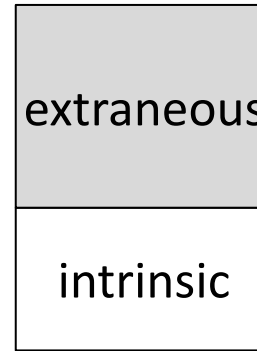
Easy-to-understand depends on "modality" first, and piped linear syntax vs. "inside-out" second

inside-out



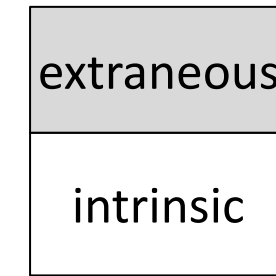
2 split points

dataflow



1 split point

inside-out



0 split points

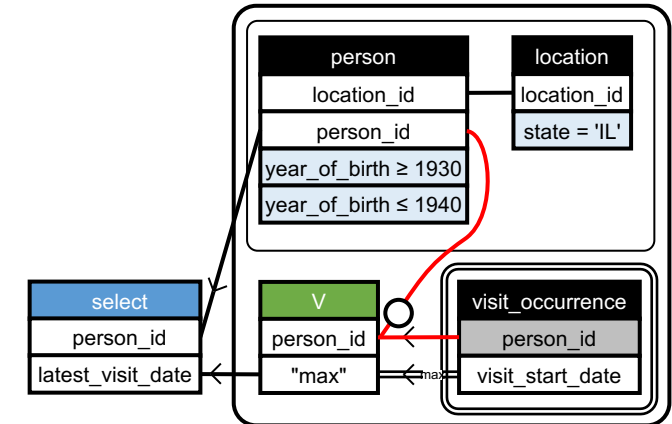
SQL (text)

```
select P.person_id, V."max" as latest_visit_date
from
  (select person_id, location_id
   from person
   where year_of_birth >= 1930
   and year_of_birth <= 1940) as P
join
  (select location_id
   from location
   where state = 'IL') as L
on P.location_id = L.location_id
left join(
  select max(visit_start_date) as "max", person_id
  from visit_occurrence
  group by person_id) as V
on P.person_id = V.person_id
```

Pipe syntax (text)

```
from person as P
|> where year_of_birth >= 1930
  and year_of_birth <= 1940
|> join location as L
  on P.location_id = L.location_id
|> where L.state = 'IL'
|> left join (
  from visit_occurrence
  |> aggregate max(visit_start_date)
  as latest_visit_date
  group by person_id) as V
on P.person_id = V.person_id
|> select P.person_id,
  V.latest_visit_date;
```

Abstract Relational Calculus (diagram)



For usability: modality instead of syntax!

Easy-to-understand depends on "modality" first, and piped linear syntax vs. "inside-out" second

inside-out

extraneous

dataflow

extraneous

inside-out

extraneous

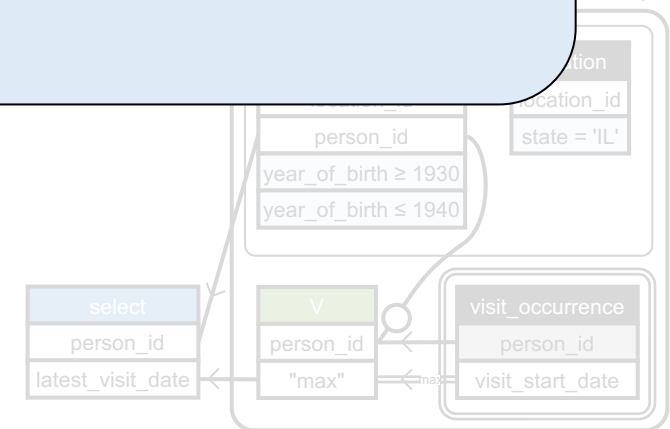
- Easy-to-understand = minimizing split points
- Dataflow has indeed fewer split points than inside-out *in text*
- But to minimize split points → go into 2D → Diagrams
= different modality, not different syntax!

SQL (text)

```
select P.person_id,
       (select max(v.visit_start_date)
        from visit_occurrence v
        where v.person_id = P.person_id
        and v.location_id = L.location_id
        and v.state = 'IL') as latest_visit_date
  from person P
  join location L
    on P.location_id = L.location_id
 where P.year_of_birth >= 1930
    and P.year_of_birth <= 1940
    and L.state = 'IL'
```

```
> join location as L
  on P.location_id = L.location_id
> where L.state = 'IL'
> left join (
  from visit_occurrence
  |> aggregate max(visit_start_date)
  as latest_visit_date
  group by person_id) as V
  on P.person_id = V.person_id
|> select P.person_id,
       V.latest_visit_date;
```

is (diagram)

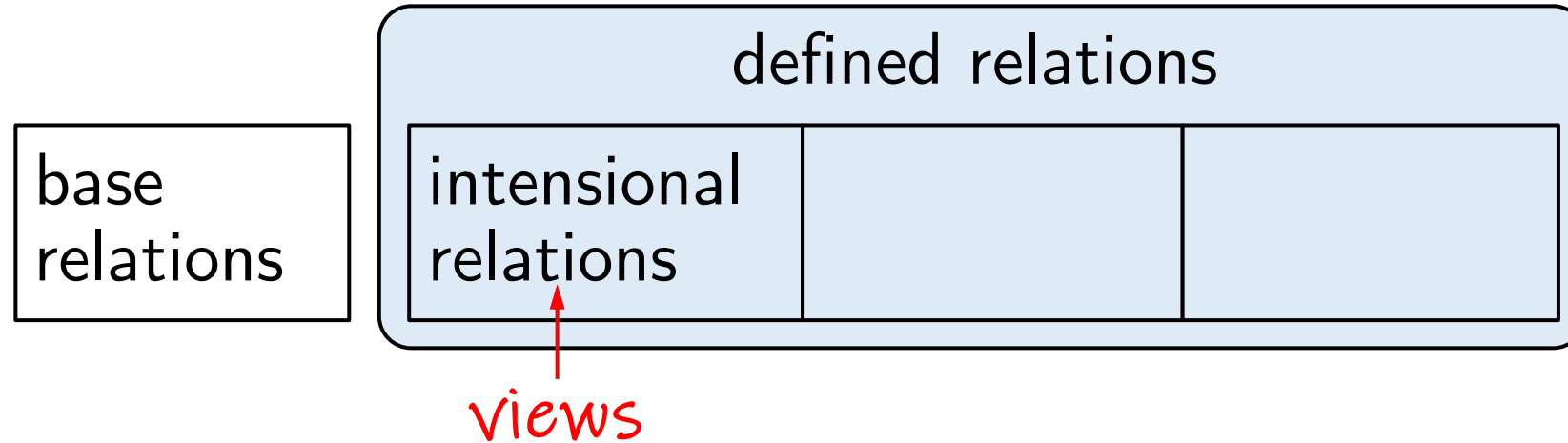


Two debates that we analyze today:

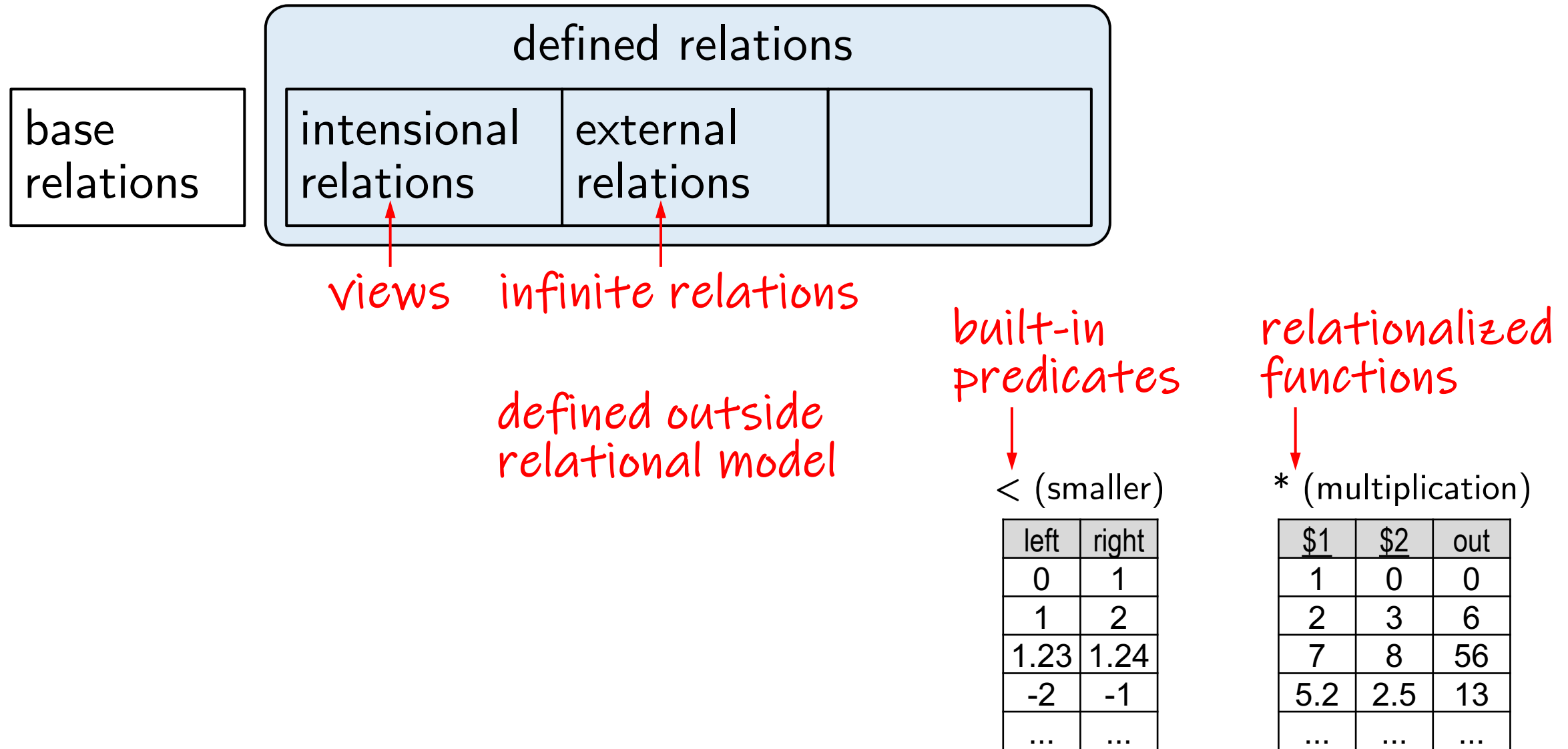
(1) Is linear dataflow inherently :
"easier to read" than inside-out?

(2) Are dataflow languages compositional, and inside-out is not?

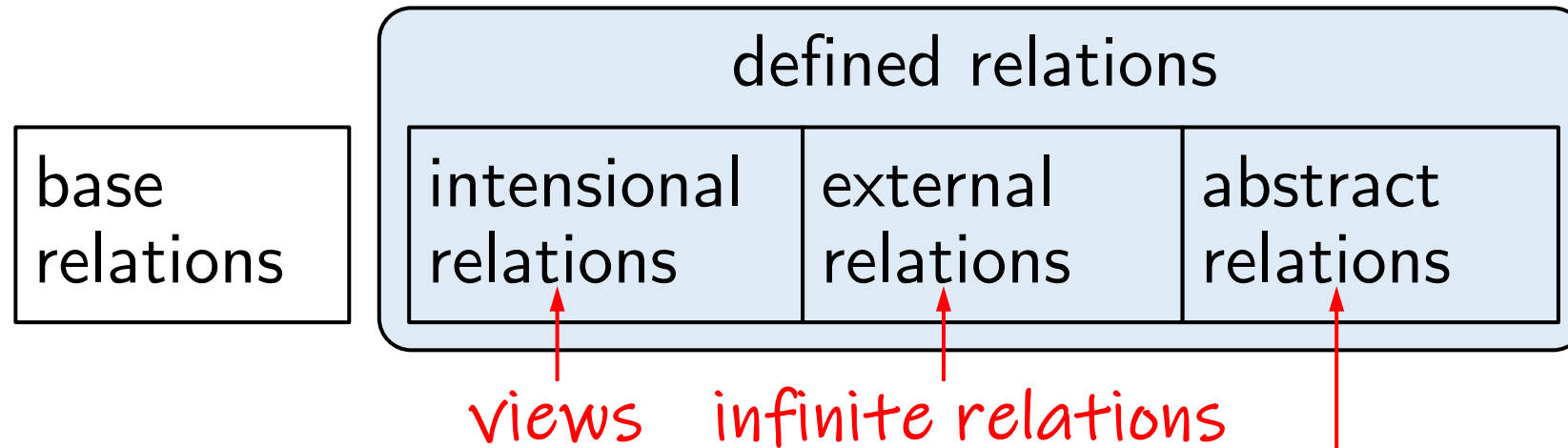
4 Types of Relations in Abstract Relational Calculus (ARC)



4 Types of Relations in Abstract Relational Calculus (ARC)



4 Types of Relations in Abstract Relational Calculus (ARC)



This is not allowed in SQL !!!

```
select L1.drinker as left,
       L2.drinker as right
where not exists
  (select 1
   from Likes L1
   and not exists
     (select 1
      from Likes L2
      where L2.drinker = L1.drinker))
```

may not have a well-defined extension on their own

defined within a relational language

serve to modularize a query

Find CIDR drinkers w/ a unique whisky taste Likes(drinker,whisky)

Alice: {1}
Bob: {1,2}
Charly: {1,2}
Dora: {1,2,3}

← not 1NF

Likes

drinker	whisky
A	1
B	1
B	2
C	1
C	2
D	1
D	2
D	3

← 1NF

?

Find CIDR drinkers w/ a unique whisky taste Likes(drinker,whisky)

```
select distinct L1.drinker
from Likes L1
where not exists
  (select 1
   from Likes L2
   where L1.drinker <> L2.drinker
   and not exists
     (select 1
      from Likes L3
      where L3.drinker = L2.drinker
      and not exists
        (select 1
         from Likes L4
         where L4.drinker = L1.drinker
         and L4.whisky = L3.whisky)))
and not exists
  (select 1
   from Likes L5
   where L5.drinker = L1.drinker
   and not exists
     (select 1
      from Likes L6
      where L6.drinker = L2.drinker
      and L6.whisky = L5.whisky)))
```

Alice: {1}
Bob: {1,2}
Charly: {1,2}
Dora: {1,2,3}

not 1NF

pipelines won't help

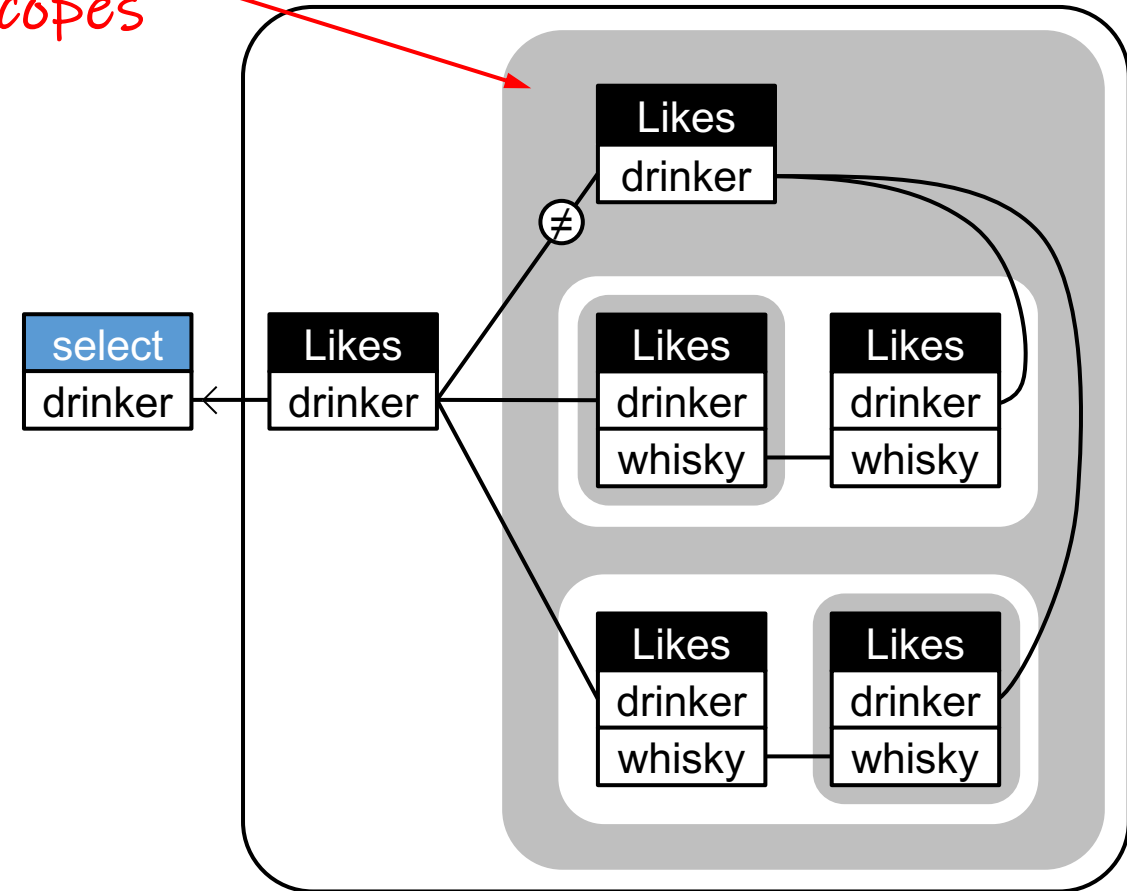
Likes

rinker	whisky
A	1
B	1
B	2
C	1
C	2
D	1
D	2
D	3

Find CIDR drinkers w/ a unique whisky taste Likes(drinker,whisky)

```
select distinct L1.drinker
from Likes L1
where not exists
  (select 1
   from Likes L2
   where L1.drinker <> L2.drinker
   and not exists
     (select 1
      from Likes L3
      where L3.drinker = L2.drinker
      and not exists
        (select 1
         from Likes L4
         where L4.drinker = L1.drinker
         and L4.whisky = L3.whisky)))
and not exists
  (select 1
   from Likes L5
   where L5.drinker = L1.drinker
   and not exists
     (select 1
      from Likes L6
      where L6.drinker = L2.drinker
      and L6.whisky = L5.whisky)))
```

*alternating shades of gray
show negation scopes*

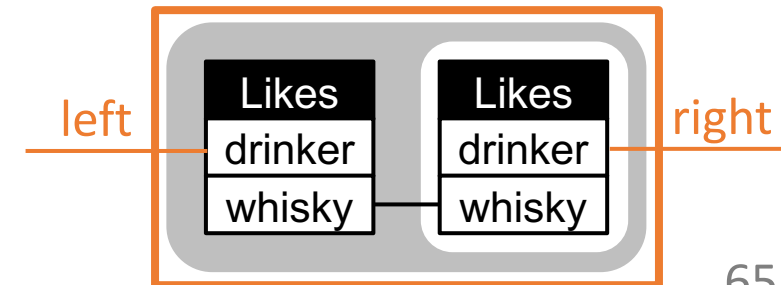
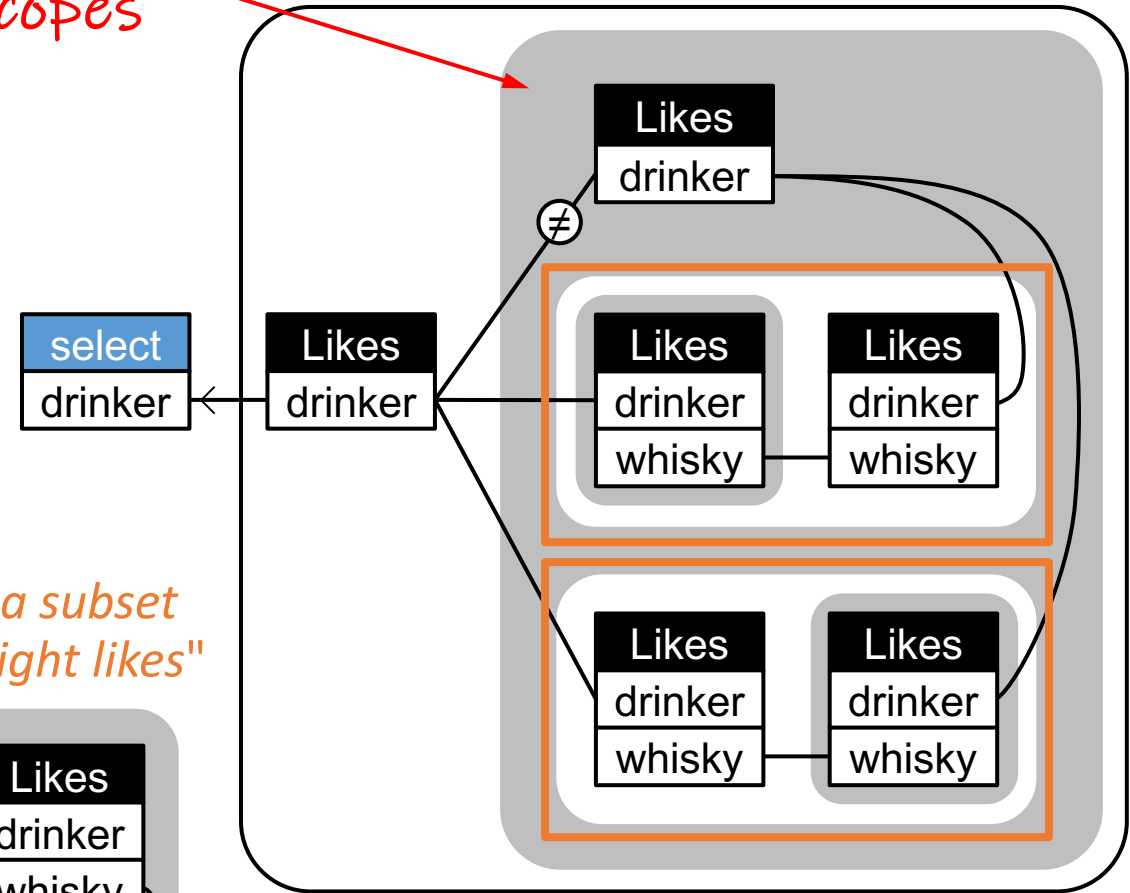
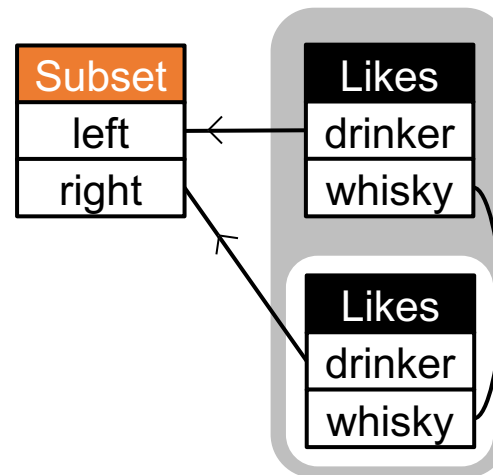


Find CIDR drinkers w/ a unique whisky taste Likes(drinker,whisky)

```
select distinct L1.drinker
from Likes L1
where not exists
  (select 1
   from Likes L2
   where L1.drinker <> L2.drinker
   and not exists
     (select 1
      from Likes L3
      where L3.drinker = L2.drinker
      and not exists
        (select 1
         from Likes L4
         where L4.drinker = L1.drinker
         and L4.whisky = L3.whisky)))
and not exists
  (select 1
   from Likes L5
   where L5.drinker = L1.drinker
   and not exists
     (select 1
      from Likes L6
      where L6.drinker = L2.drinker
      and L6.whisky = L5.whisky)))
```

alternating shades of gray
show negation scopes

"left drinker likes a subset
of whiskies that right likes"

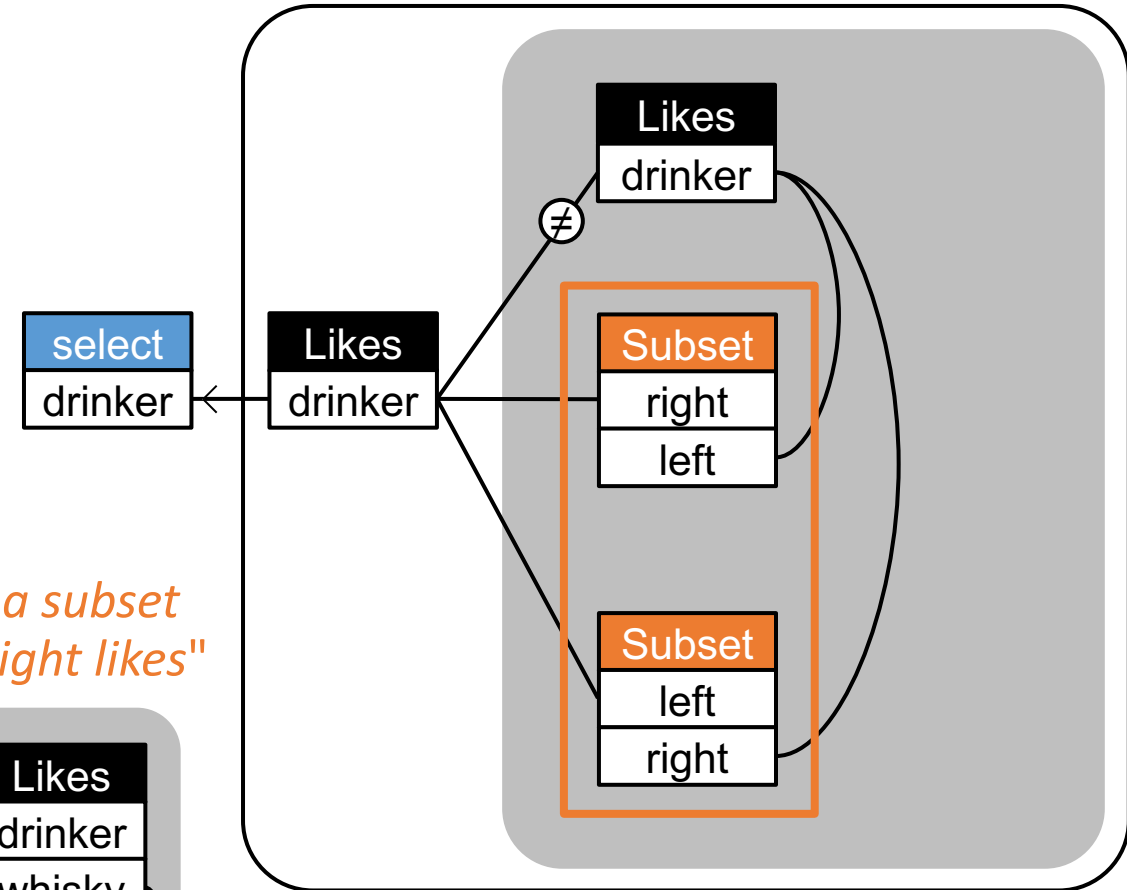
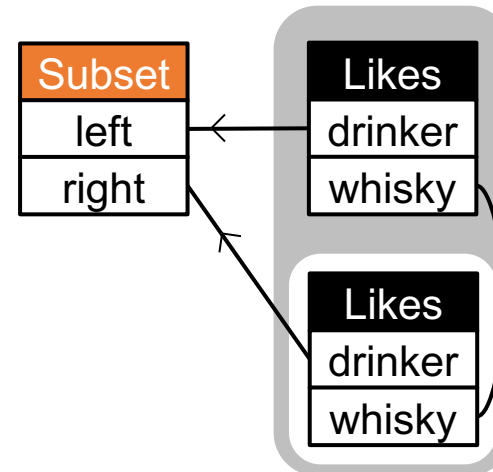


Find CIDR drinkers w/ a unique whisky taste Likes(drinker,whisky)

```
select distinct L1.drinker
from Likes L1
where not exists
(
  select 1
  from Likes L2, Subset S1, Subset S2
  where L1.drinker <> L2.drinker
  and S1.left = L2.drinker
  and S1.right = L1.drinker
```

```
and S1.left = L1.drinker
and S1.right = L2.drinker
)
```

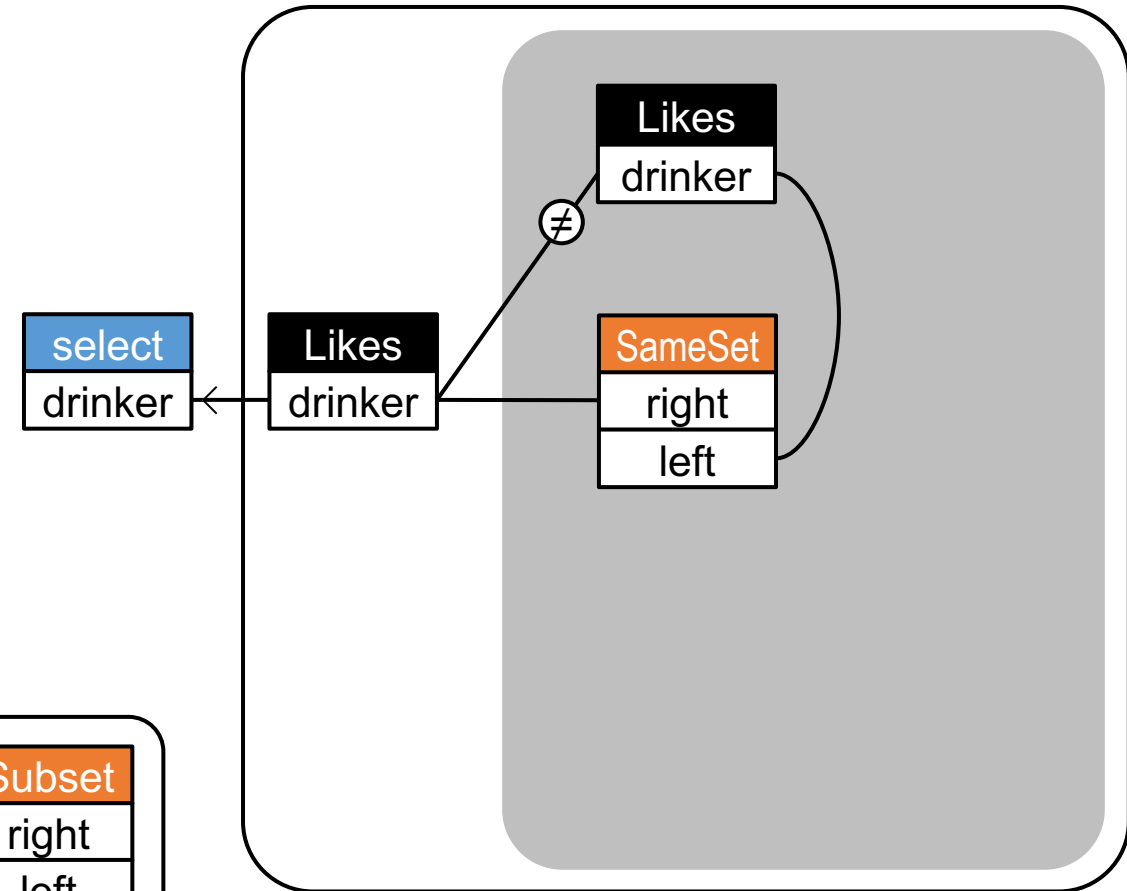
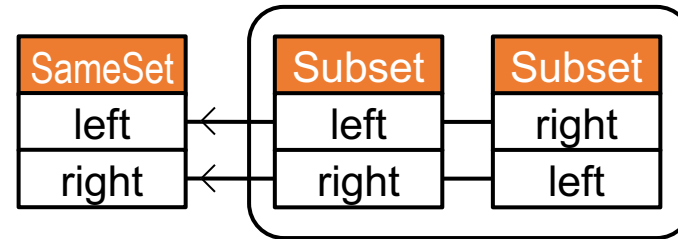
"left drinker likes a subset of whiskies that right likes"



Find CIDR drinkers w/ a unique whisky taste Likes(drinker,whisky)

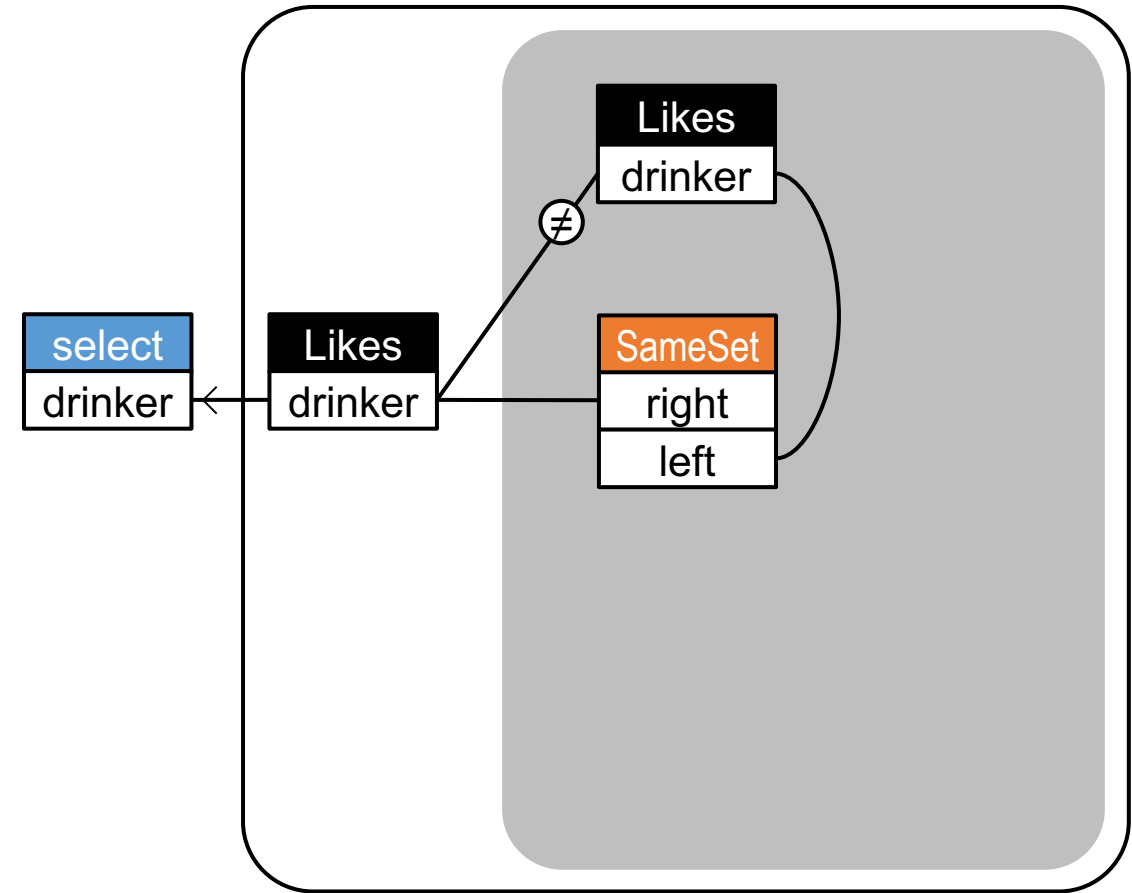
```
select distinct L1.drinker
from Likes L1
where not exists
  (select 1
   from Likes L2, SameSet S
   where L1.drinker <> L2.drinker
   and S.left = L2.drinker
   and S.right = L1.drinker)
```

"pairs of drinkers who like the same set of whiskies"



Find CIDR drinkers w/ a unique whisky taste Likes(drinker,whisky)

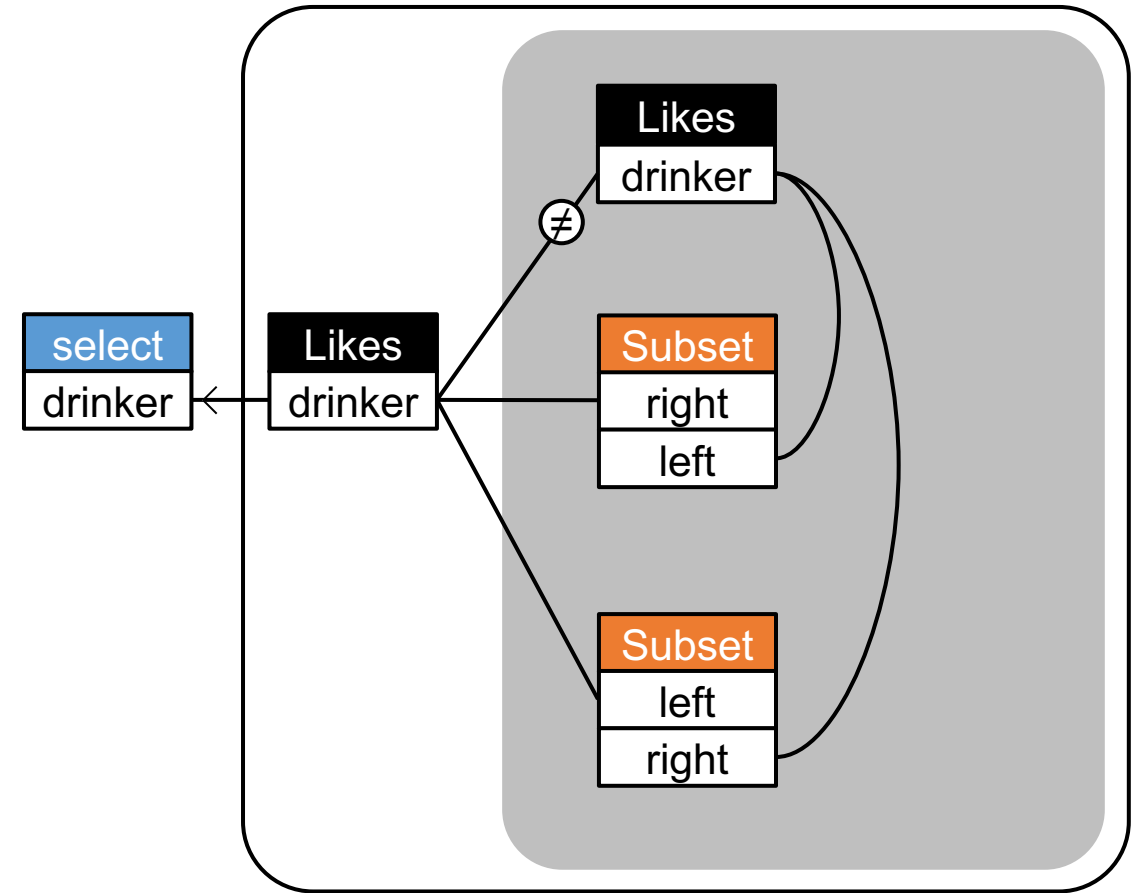
```
select distinct L1.drinker
from Likes L1
where not exists
  (select 1
   from Likes L2, SameSet S
   where L1.drinker <> L2.drinker
        and S.left = L2.drinker
        and S.right = L1.drinker)
```



Find CIDR drinkers w/ a unique whisky taste Likes(drinker,whisky)

```
select distinct L1.drinker
from Likes L1
where not exists
  (select 1
   from Likes L2, Subset S1, Subset S2
   where L1.drinker <> L2.drinker
   and S1.left = L2.drinker
   and S1.right = L1.drinker
```

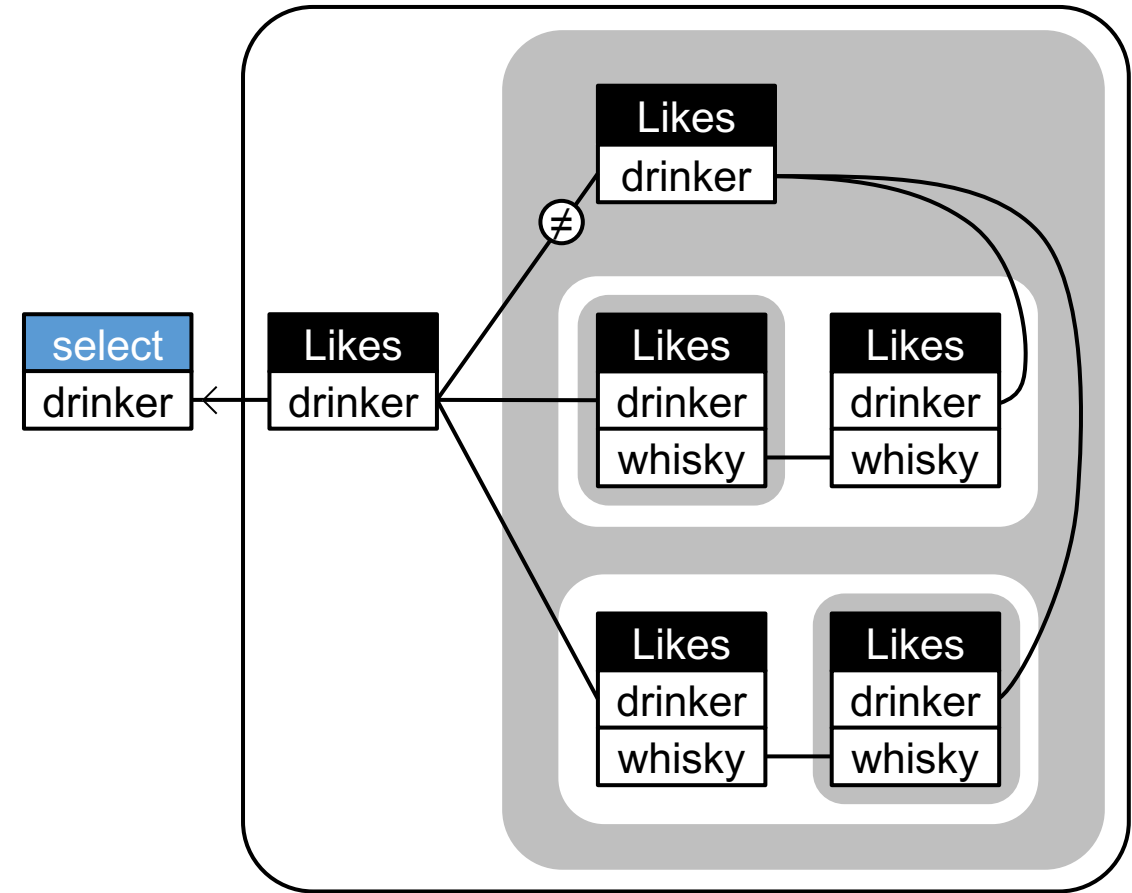
```
and S1.left = L1.drinker
and S1.right = L2.drinker
```



)

Find CIDR drinkers w/ a unique whisky taste Likes(drinker,whisky)

```
select distinct L1.drinker
from Likes L1
where not exists
  (select 1
   from Likes L2
   where L1.drinker <> L2.drinker
   and not exists
     (select 1
      from Likes L3
      where L3.drinker = L2.drinker
      and not exists
        (select 1
         from Likes L4
         where L4.drinker = L1.drinker
         and L4.whisky = L3.whisky)))
and not exists
  (select 1
   from Likes L5
   where L5.drinker = L1.drinker
   and not exists
     (select 1
      from Likes L6
      where L6.drinker = L2.drinker
      and L6.whisky = L5.whisky)))
```

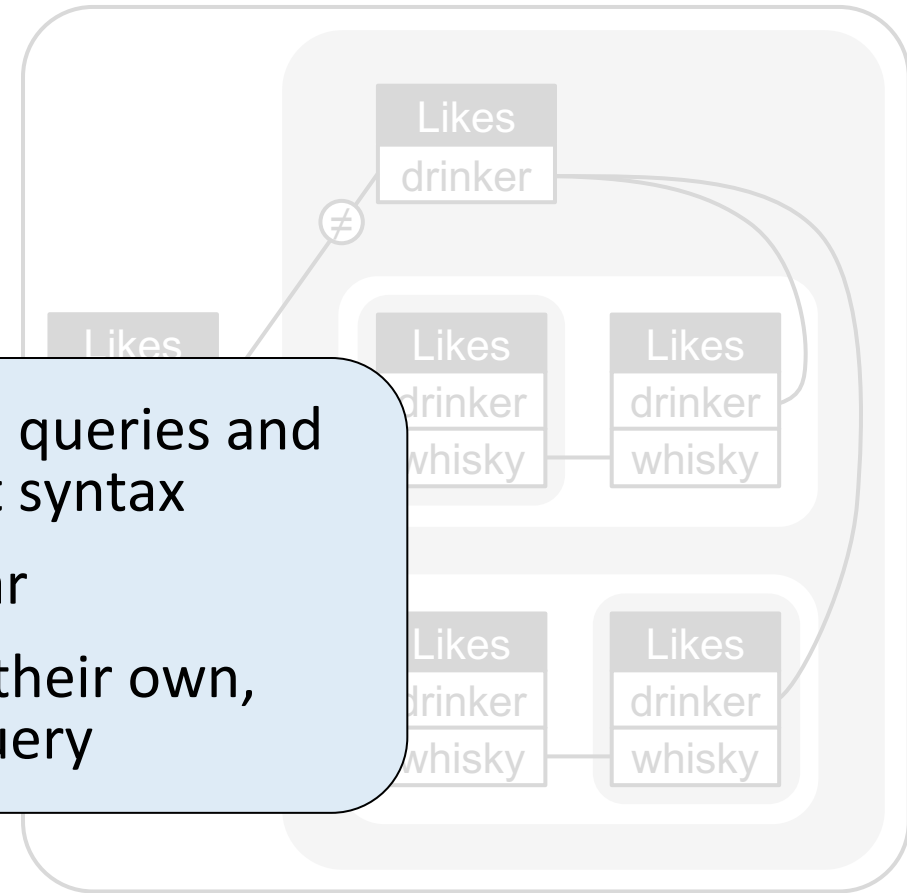


Find CIDR drinkers w/ a unique whisky taste Likes(drinker,whisky)

```
select distinct L1.drinker
from Likes L1
where not exists
  (select 1
   from Likes L2
   where L1.drinker <> L2.drinker
   and not exists
```

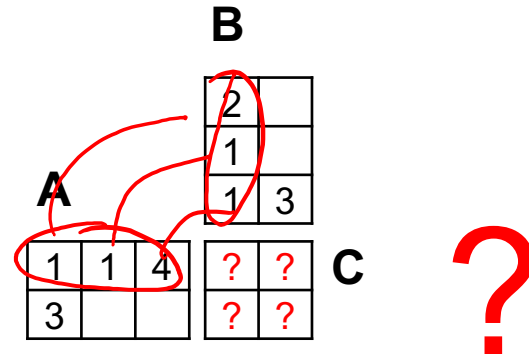
- Abstract relations can "modularize" complicated queries and allow "collapse and expand", even for inside-out syntax
 - no need for pipes to make programs modular
- They may not have a well-defined extension on their own, but are well defined within the context of the query

```
  (select 1
   from Likes L2
   where L1.drinker <> L2.drinker
   and not exists
    (select 1
     from Likes L3
     where L2.drinker = L3.drinker
     and L2.whisky = L3.whisky)))
where L5. drinker = L1. drinker
and not exists
  (select 1
   from Likes L6
   where L6.drinker = L2.drinker
   and L6.whisky = L5.whisky)))
```



An example: Matrix Multiplication

Matrix Multiply $C = A \cdot B$



Matrix Multiply $C = A \cdot B$

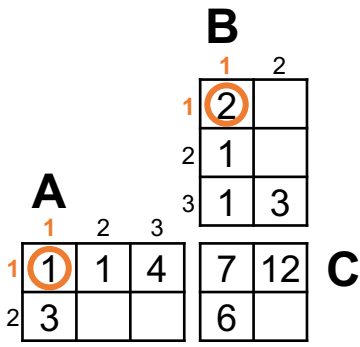
			B	
			2	
			1	
			1	3
A			7	12
1	1	4	6	
3				
			C	

Matrix Multiply $C = A \cdot B$

A			B		C
1	2	3	1	2	
1	1	4	2		
2	3		1		
			1	3	
			7	12	
			6		

A			B		
row	col	val	row	col	val
1	1	1	1	1	2
1	2	1	2	1	1
1	3	4	3	1	1
2	1	3	3	2	3

Matrix Multiply $C = A \cdot B$



input

A			B		
row	col	val	row	col	val
1	1	1	1	1	2
1	2	1	2	1	1
1	3	4	3	1	1
2	1	3	3	2	3

Matrix Multiply $C = A \cdot B$



SQL

```
select A.row, B.col, sum(A.val*B.val) val
from A, B
where A.col = B.row
group by A.row, B.col
```

Diagram illustrating the matrix multiplication process. Matrix A (3x3) and Matrix B (3x2) are shown, along with the resulting matrix C (3x2). The calculation for the element at row 1, column 1 of C is highlighted: $1 \cdot 2 + 1 \cdot 1 + 4 \cdot 1 = 7$.

A			B		C
1	2	3	1	2	
1	1	4	2	1	
2	3	6	3	1	

input

A			B		
row	col	val	row	col	val
1	1	1	1	1	2
1	2	1	2	1	1
1	3	4	3	1	1
2	1	3	3	2	3

equijoin

a.row	a.col	a.val	b.row	b.col	b.val	a.val*b.val
1	1	1	1	1	2	2
1	2	1	2	1	1	1
1	3	4	3	1	1	4
1	3	4	3	2	3	12
2	1	3	1	1	2	6

group by (a.row,b.col)

	a.row	a.col	a.val	b.row	b.col	b.val	a.val*b.val
(1,1) → {	1	1	1	1	1	2	2
	1	2	1	2	1	1	1
	1	3	1	3	1	1	4
(1,2) → {	1	3	1	3	2	1	12
(2,1) → {	2	1	3	1	1	2	6

aggregation

a.row	b.col	sum(a.val*b.val)
1	1	7
1	2	12
2	1	6

Matrix Multiply $C = A \cdot B$



SQL

```
select A.row, B.col, sum(A.val*B.val) val
from A, B
where A.col = B.row
group by A.row, B.col
```

```
select A.row, B.col, sum(M.out) val
from A, B, M
where A.col = B.row
and A.val = M.v1
and B.val = M.v2
group by A.row, B.col
```

Diagram showing matrices A, B, and C with row and column indices.

A

1	1	4
3		

B

2	
1	
1	3

C

7	12
6	

input

A			B			M(multiplication)		
row	col	val	row	col	val	m.v ₁	m.v ₂	m.out
1	1	1	1	1	2	1	1	1
1	2	1	2	1	1	1	2	2
1	3	4	3	1	1	3	2	6
2	1	3	3	2	3	4	1	4
						4	3	12

equijoin

a.row	a.col	a.val	b.row	b.col	b.val	m.v ₁	m.v ₂	m.out
1	1	1	1	1	2	1	2	2
1	2	1	2	1	1	1	1	1
1	3	4	3	1	1	4	1	4
1	3	4	3	2	3	4	3	12
2	1	3	1	1	2	3	2	6

group by (a.row,b.col)

a.row	a.col	a.val	b.row	b.col	b.val	m.v ₁	m.v ₂	m.out
1	1	1	1	1	2	1	2	2
1	2	1	2	1	1	1	1	1
1	3	1	3	1	1	4	1	4
1	3	1	3	2	1	4	3	12
2	1	3	1	1	2	3	2	6

aggregation

a.row	b.col	sum(m.out)
1	1	7
1	2	12
2	1	6

Matrix Multiply $C = A \cdot B$

SQL

```
select A.row, B.col, sum(A.val*B.val) val
from A, B
where A.col = B.row
group by A.row, B.col
```

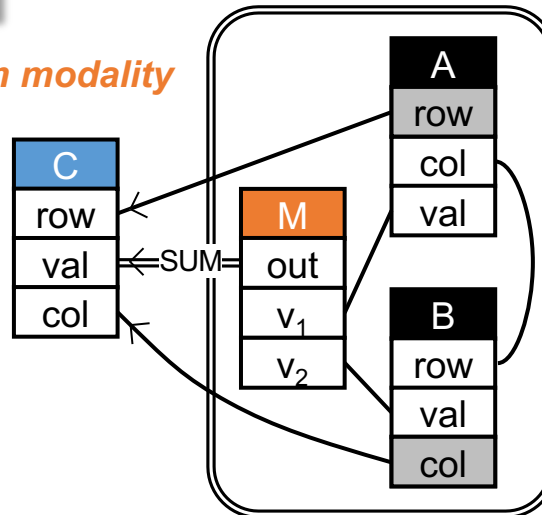
```
select A.row, B.col, sum(M.out) val
from A, B, M
where A.col = B.row
and A.val = M.v1
and B.val = M.v2
group by A.row, B.col
```

Abstract Relational Calculus (ARC)

Comprehension Syntax

```
{C(row,col,val) |  $\gamma a \in A, b \in B, m \in M,$ 
  group by({a.row, b.col})
  [C.row=a.row  $\wedge$  C.col=b.col  $\wedge$  C.val=sum(m.out)
  m.v1=a.val  $\wedge$  m.v2=b.val] }
```

Diagram modality



input

A			B			M(ultiplication)		
row	col	val	row	col	val	m.v ₁	m.v ₂	m.out
1	1	1	1	1	2	1	1	1
1	2	1	2	1	1	1	2	2
1	3	4	3	1	1	3	2	6
2	1	3	3	2	3	4	1	4
						4	3	12

equijoin

a.row	a.col	a.val	b.row	b.col	b.val	m.v ₁	m.v ₂	m.out
1	1	1	1	1	2	1	2	2
1	2	1	2	1	1	1	1	1
1	3	4	3	1	1	4	1	4
1	3	4	3	2	3	4	3	12
2	1	3	1	1	2	3	2	6

group by
(a.row,b.col)

a.row	a.col	a.val	b.row	b.col	b.val	m.v ₁	m.v ₂	m.out
(1,1) →	1	1	1	1	2	1	2	2
	1	2	2	1	1	1	1	1
	1	3	3	1	1	4	1	4
(1,2) →	1	3	1	3	2	4	3	12
(2,1) →	2	1	3	1	2	3	2	6

aggregation

a.row	b.col		sum(m.out)
1	1		7
1	2		12
2	1		6

Shortest two-hop path

SQL

```
select A.row, B.col, min(A.val+B.val) val
from A, B
where A.col = B.row
group by A.row, B.col
```

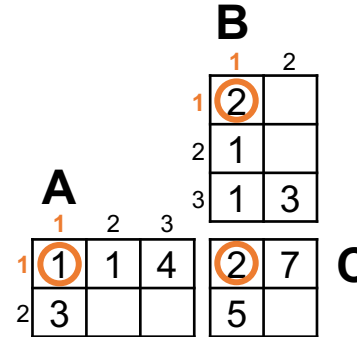
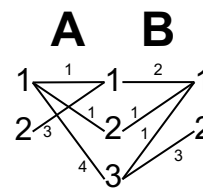
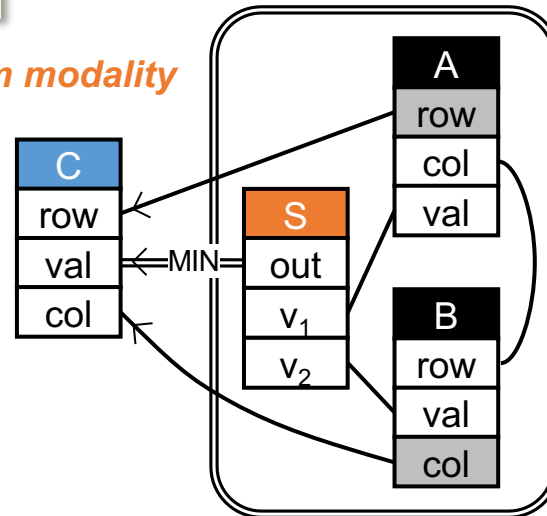
```
select A.row, B.col, min(S.out) val
from A, B, S
where A.col = B.row
and A.val = S.v1
and B.val = S.v2
group by A.row, B.col
```

Abstract Relational Calculus (ARC)

Comprehension Syntax

$\{C(\text{row}, \text{col}, \text{val}) \mid \gamma a \in A, b \in B, s \in S,$
 $\text{group by}(\{a.\text{row}, b.\text{col}\})$
 $[C.\text{row} = a.\text{row} \wedge C.\text{col} = b.\text{col} \wedge C.\text{val} = \min(s.\text{out})$
 $s.v_1 = a.\text{val} \wedge s.v_2 = b.\text{val}] \}$

Diagram modality



A			B			S(um)		
row	col	val	row	col	val	s.v ₁	s.v ₂	s.out
1	1	1	1	1	2	1	1	2
1	2	1	2	1	1	1	2	3
1	3	4	3	1	1	3	2	5
2	1	3	3	2	3	4	1	5
						4	3	7

equijoin

a.row	a.col	a.val	b.row	b.col	b.val	s.v ₁	s.v ₂	s.out
1	1	1	1	1	2	1	2	3
1	2	1	2	1	1	1	1	2
1	3	4	3	1	1	4	1	5
1	3	4	3	2	3	4	3	7
2	1	3	1	1	2	3	2	5

group by
(a.row, b.col)

a.row	a.col	a.val	b.row	b.col	b.val	s.v ₁	s.v ₂	s.out
1	1	1	1	1	2	1	2	3
1	2	1	2	1	1	1	1	2
1	3	1	3	1	1	4	1	5
1	3	1	3	2	1	4	3	7
2	1	3	1	1	2	3	2	5

aggregation

a.row	b.col		min(s.out)
1	1		2
1	2		7
2	1		5

In Semiring Framework

SQL

```
select A.row, B.col,  $\oplus(A.val \otimes B.val)$  val
from A, B
where A.col = B.row
group by A.row, B.col
```

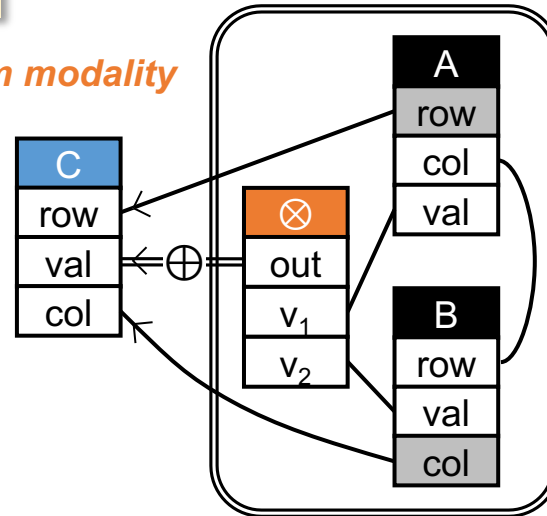
```
select A.row, B.col,  $\oplus(P.out)$  val
from A, B,  $\otimes$  as P
where A.col = B.row
and A.val = P.v1
and B.val = P.v2
group by A.row, B.col
```

Abstract Relational Calculus (ARC)

Comprehension Syntax

```
{C(row,col,val) |  $\gamma a \in A, b \in B, p \in \otimes,$   
group by({a.row, b.col})  
[C.row=a.row  $\wedge$  C.col=b.col  $\wedge$  C.val= $\oplus(p.out)$   
p.v1=a.val  $\wedge$  p.v2=b.val] }
```

Diagram modality



input

A			B			P ($\otimes$)		
row	col	val	row	col	val	p.v ₁	p.v ₂	p.out
1	1	1	1	1	2	1	1	...
1	2	1	2	1	1	1	2	...
1	3	4	3	1	1	3	2	...
2	1	3	3	2	3	4	1	...
						4	3	...

equijoin

a.row	a.col	a.val	b.row	b.col	b.val	p.v ₁	p.v ₂	p.out
1	1	1	1	1	2	1	2	...
1	2	1	2	1	1	1	1	...
1	3	4	3	1	1	4	1	...
1	3	4	3	2	3	4	3	...
2	1	3	1	1	2	3	2	...

group by
(a.row, b.col)

a.row	a.col	a.val	b.row	b.col	b.val	p.v ₁	p.v ₂	p.out
(1,1) →	1	1	1	1	2	1	2	...
	1	2	2	1	1	1	1	...
	1	3	3	1	1	4	1	...
(1,2) →	1	3	1	3	2	4	3	...
(2,1) →	2	1	3	1	2	3	2	...

aggregation

a.row	b.col	min(s.out)
1	1	...
1	2	...
2	1	...

Take-aways

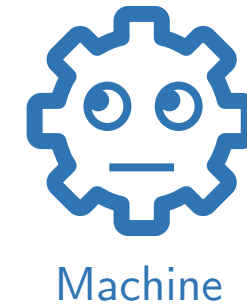
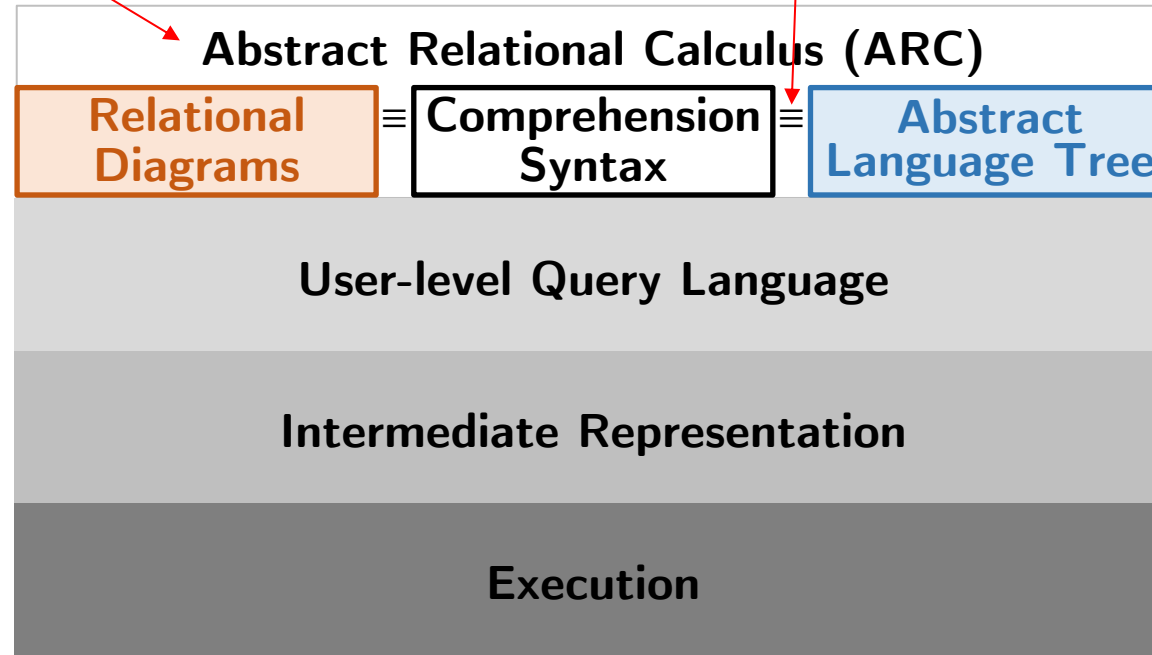
An Abstract Reference Language with refined vocabulary

Reference language. Focus is on the intent: Separates "relational intent" from representation.

modality: alternative representation of the intent tailored to different audiences



Human



Machine

more
abstract

convention: orthogonal environment-level semantic parameters under which the relational intent is interpreted (e.g., set vs. bag semantics, or treatment of null values)

Choose an (optional) example:

Select example...

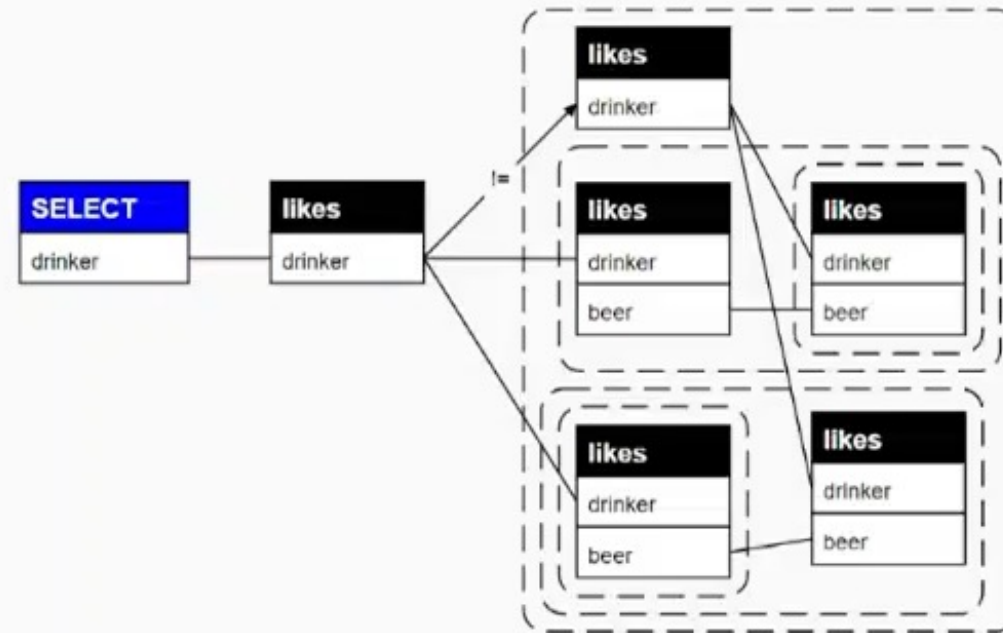
Enter a schema:

Likes (drinker, beer)

Enter a SQL query:

```
select distinct L1.drinker
from Likes as L1
where not exists
    (select 1
     from Likes as L2
     where L1.drinker <>
L2.drinker
    and not exists
        (select 1
         from Likes as L3
         where L3.drinker =
L2.drinker
        and not exists
            (select 1
```

Submit



Take-aways

- Abstract Relational Calculus: strict generalization of Tuple RC
 - intended as reference language, separating relational intent / modality (representation for audience) / convention (e.g., sets or bags)
- Most relevant papers (all available from RelationalDiagrams.com)
 - VLDB'23/ICDE'24: 500+ slide tutorial on diagrammatic representations
 - SIGMOD'24/Sigmod record'25: showing calculus can express more 'relational patterns' than algebra / reproducible user study
 - CIDR'26: today's talk: extensions to a collection framework
 - SIGMOD'26: disjunctions in diagrams
- Next steps
 - theory: formal proofs / nested relational model / sorting
 - practice: transpilers from other languages / NL2SQL

Thanks 😊

